

C O D E R H O U S E
M A T E R I A L D E L E C T U R A

AI Automation

Guía de lectura del curso

Contenido

MÓDULO 1 • Introducción a la IA, el Prompt Engineering y las automatizaciones	4
¿Qué es la Inteligencia Artificial?	4
Cómo procesa el texto un LLM: tokenizador, tokens y ventana de contexto	5
Prompt Engineering: anatomía de un prompt efectivo	6
Repaso de conceptos del módulo	9
MÓDULO 2 • Introducción a las automatizaciones y n8n	10
¿Qué es una automatización?	10
ROI, escalabilidad y estandarización de procesos	11
¿Qué es n8n? Instancias, ejecución local y en la nube	12
Importar y reutilizar templates	13
PRE-ENTREGA Preentrega 1: De la idea al prompt	15
MÓDULO 3 • Conceptos básicos	17
Anatomía de un workflow: nodos, conexiones y flujo de datos	17
Nodos principales en n8n	18
MÓDULO 4 • Conceptos técnicos	20
¿Qué es una API? Modelo cliente-servidor y endpoints	20
¿Qué es HTTP?	21
Credenciales y autenticación: API Keys y OAuth2	22
PRE-ENTREGA Preentrega 2: Infraestructura de integración técnica	24
MÓDULO 5 • Creando workflows con aplicaciones y servicios externos	26
Planificar un workflow antes de construirlo: entrada, proceso y salida	26
Conectando servicios de Google	27
MÓDULO 6 • Implementación del AI Agent	28
¿Qué es un AI Agent? Diferencias con un nodo de IA simple	28
El system prompt del agente: instrucciones y límites	30
PRE-ENTREGA Preentrega 3: Planificá y construí tu primer AI Agent	32
MÓDULO 7 • Instalando IA local con Ollama	35
Por qué ejecutar un modelo local: costo, privacidad e independencia	35
Ollama y descarga de modelos	36

MÓDULO 8 • WhatsApp Business	38
Cómo funciona la WhatsApp Business Platform (Cloud API)	38
WhatsApp Business + AI Agent + Prompt Engineering	39
PRE-ENTREGA Preentrega 4: Ensamblaje del asistente de WhatsApp e IA	41
MÓDULO 9 • Proyecto Final: despliegue de agente autónomo cognitivo	44
ENTREGA FINAL Proyecto Final: asistente autónomo de WhatsApp desplegado	45
MATERIAL COMPLEMENTARIO • Glosario, análisis y cierre del curso	49
Palabras clave	49
La transición de flujos lineales a agentes cognitivos	50
El arte de la salida estructurada: por qué el JSON es el lenguaje de la libertad en IA	51
IA local vs. cloud: el equilibrio entre privacidad, costo y potencia con Ollama	51
Preguntas frecuentes	52
Conclusiones	54

Introducción a la IA, el Prompt Engineering y las automatizaciones

En este primer módulo aprenderás los fundamentos básicos de IA y cuáles son las técnicas de optimización de prompts para obtener datos estructurados.

¿Qué es la Inteligencia Artificial?

Conocé a qué nos referimos cuando hablamos de Inteligencia Artificial, cuáles son sus subcampos y qué es un Large Language Model (LLM).

Cuando hablamos de Inteligencia Artificial, lo primero que se nos viene a la cabeza es Claude, ChatGPT o alguno de estos. Si bien es aceptable llamarlos de tal manera, si nos basamos en terminología académica no es correcto, ya que la Inteligencia Artificial es una **rama de la informática**, como lo es la ciberseguridad, la ciencia de datos o la ingeniería de software. Así como cada una de estas ramas tiene un objetivo específico, **la Inteligencia Artificial busca desarrollar sistemas capaces de realizar tareas que necesiten de inteligencia humana**.

A su vez, dentro de la Inteligencia Artificial tenemos más subcampos, como por ejemplo:

- **Machine Learning (Aprendizaje Automático):** es bastante amplio este campo, es el encargado de que las máquinas aprendan en base a ejemplos, identificando patrones sin que se les diga qué es lo que deberían de hacer. Cuantos más datos reciben, mejores son sus resultados.
- **Deep Learning (Aprendizaje Profundo):** es un subcampo del Machine Learning, usa estructuras llamadas redes neuronales con múltiples capas para resolver problemas complejos.
- **NLP (Natural Language Processing):** es una rama que permite a las computadoras leer, interpretar, comprender y generar el lenguaje humano, ya sea escrito o hablado.
- **Visión Artificial (Computer Vision):** si el NLP se encarga del hablar y escribir, esta rama se enfoca en entender imágenes y videos.

Ya teniendo idea de estos conceptos, sabemos que el Deep Learning usa redes neuronales con múltiples capas para problemas complejos, y que el NLP se enfoca en que las máquinas entiendan el lenguaje humano. Bien. **¿Qué pasa cuando combinamos ambas cosas y las entrenamos con prácticamente todo el texto que existe en internet?** Obtenemos lo que se llama un **Large Language Model**, un LLM. ChatGPT es un LLM. Claude es un LLM. Estos son los modelos que van a vivir dentro de nuestras automatizaciones.

¿Qué es un Large Language Model (LLM)?

Es un sistema diseñado para comprender, procesar y generar lenguaje humano, basado en una red neuronal entrenada con enormes cantidades de texto. Para que los LLM logren una fluidez de respuesta es gracias a que fueron entrenados con cantidades masivas de datos sacados de Internet (libros, fo-

ros, artículos). Por lo tanto, tienen la capacidad de identificar patrones: aprenden qué palabras suelen seguir a otras, cómo estructurar frases, entre otro tipo de cosas.

Toda respuesta que nos proporciona un LLM es básicamente por probabilidades de palabras seguidas.

Ejemplo

Si son las 21:00, evidentemente sabemos que luego de un "Hola, buenas ____" viene noches, ya que el contexto horario favorece a que sea esa la respuesta. Bueno, en un LLM, el modelo calcula qué palabra es la adecuada y con mayor probabilidad de seguir en dicha oración.

El LLM no "entiende" lo que le decimos de la misma forma que un humano, simplemente leyó tanta documentación gracias al Deep Learning que realiza un mapeo matemático complejo para darnos una respuesta, expresándola de formas muy naturales. En resumen, simula el comportamiento humano calculando qué palabra debería ir después de otra.

Cómo procesa el texto un LLM: tokenizador, tokens y ventana de contexto

Comprenderás cómo un LLM procesa el texto, qué es un token y de qué hablamos cuando decimos que la IA puede "alucinar".

Ahora comprendemos que cuando le mandamos un mensaje a un LLM, el modelo realiza el proceso matemático de calcular las probabilidades de la siguiente palabra, usando su red neuronal y su mecanismo de atención para decidir estadísticamente que después de "Buenos" posiblemente venga "Días". Eso sería una *inferencia*. La *inferencia* en un modelo de lenguaje grande (LLM) es la fase de ejecución donde la red neuronal, ya entrenada, aplica sus conocimientos matemáticos para predecir y generar el siguiente fragmento de texto basándose estrictamente en el contexto que recibe. Todo esto está basado en el verdadero lenguaje del LLM: **los tokens**.

Tokens

Los LLM no usan palabras completas para hacer estas predicciones, sino que usan tokens. Un token puede ser una palabra entera, una sílaba, o incluso una sola letra. Todo este proceso de los tokens es gracias al *tokenizador*: un algoritmo de compresión y segmentación de texto. Los LLM modernos no utilizan separación por espacios, sino que utilizan algoritmos estadísticos de subpalabras, siendo el más común el BPE (Byte-Pair Encoding).

Importante

El token representa una unidad de texto. Cada modelo está entrenado de una forma distinta, usa tokenizaciones distintas y utiliza (matemáticamente hablando) otro tipo de asociaciones, por eso **no todos los modelos son buenos para todos**.

Ventana de contexto

Existe un límite a la cantidad de tokens que un LLM puede "tener en mente" al mismo tiempo, a esto se le llama **ventana de contexto**. La idea de hacer buenos prompts es justamente no gastar este contexto y la capacidad de cómputo que tiene el modelo para respondernos correctamente. No es que el modelo "olvida de a poco", como una persona, sino que **tiene un límite fijo de tokens que puede procesar a la vez**. Una vez que se supera ese límite, lo que quedó "afuera" de la ventana simplemente deja de estar disponible para el modelo. Es como una hoja de papel de tamaño fijo: si queremos seguir escribiendo y ya no hay espacio, deberíamos de borrar el principio.

Alucinaciones

Esto conecta directamente con las **alucinaciones**: si el modelo "perdió" información clave porque ya no entra en su ventana de contexto, en lugar de decir "no tengo esa información", puede inventar una respuesta plausible para completar el vacío, con total confianza, aunque esté equivocado. No solamente pasa con la ventana de contexto, sino también con información que quizás no le brindamos.

Ejemplo

Imaginen a un estudiante que va a un examen para el que no estudió nada. Cuando el profesor le hace una pregunta, el estudiante, en lugar de decir "no sé", empieza a inventar una respuesta larguísima, usando palabras muy sofisticadas, con muchísima seguridad en sí mismo. Suena tan convincente que casi le creés.

Eso es una alucinación de la IA. Es cuando **el modelo genera información falsa, inventada o sin sentido**, pero la presenta como si fuera una verdad absoluta.

Prompt Engineering: anatomía de un prompt efectivo

Esta unidad explica qué es el Prompt Engineering: la práctica de diseñar instrucciones claras y específicas para obtener mejores respuestas de una IA. Desarma la anatomía de un prompt efectivo (rol, tarea, contexto y formato de salida), muestra para qué sirven los delimitadores cuando las instrucciones y el dato a procesar viajan en el mismo prompt, y presenta dos técnicas clave: Zero-Shot Prompting y Few-Shot Prompting.

¿Qué es el Prompt Engineering o Ingeniería de Prompts?

El **Prompt Engineering** es una **práctica que busca crear de forma metódica las instrucciones o preguntas que se le van a hacer a un modelo**. Todo esto es con el fin de obtener respuestas precisas y correctas, alineadas con lo que nosotros le estamos solicitando. Estas instrucciones/preguntas son los famosos "prompts".

Un prompt es el **punto de comunicación entre nosotros y la Inteligencia Artificial**, es decir, es el texto, la frase o conjunto de instrucciones específicas que le pedimos a la IA con el fin de obtener un resultado. Si nosotros no trabajamos con buenos prompts, la IA no va a comprender bien las instruccio-

nes que le estamos dando, por lo que esto mismo afectará la respuesta que se obtendrá y, por ende, no nos va a retornar lo que le estamos pidiendo o, en caso que lo haga, no va a ser de forma eficiente.

Ejemplo

Si le digo a la IA "Armame una rutina para el gym", nos va a devolver algo súper genérico. En cambio, si nos tomamos el tiempo de desarrollar un prompt que sea bien estructurado, especificándole el grupo muscular, intensidad y demás, evidentemente se va a obtener una respuesta mejor.

Entonces, un prompt no es solamente hacerle una pregunta a la IA, es darle contexto/narrativa y ser lo más específico posible. Todo esto con el fin de sacarle el máximo provecho y obtener los mejores resultados posibles. Sabiendo ahora que una buena redacción o un pequeño cambio en un prompt puede cambiar completamente la respuesta que obtenemos, veamos unos conceptos para que los tengan en cuenta.

La anatomía de un prompt efectivo

Un prompt que funciona no es más largo, es más ordenado. La mayoría de los prompts efectivos tienen cuatro componentes:

- **Rol:** quién queremos que sea la IA al responder. *"Sos un asistente de atención al cliente de una tienda de indumentaria online."*
- **Tarea:** qué tiene que hacer, con un verbo concreto y sin ambigüedad. *"Clasificá el mensaje en una de estas tres categorías."*
- **Contexto:** la información que necesita para resolverlo y que no puede adivinar. *"Los envíos demoran 5 días hábiles; un pedido no es un reclamo hasta pasados esos 5 días."*
- **Formato de salida:** cómo queremos la respuesta, sobre todo si va a seguir viajando por una automatización. *"Devolvé únicamente la categoría, en minúscula, sin explicaciones."*

Los cuatro no son obligatorios siempre, pero cuando el resultado sale genérico o inconsistente, casi siempre falta alguno.

Los delimitadores

Cuando en un mismo prompt conviven **nuestras instrucciones** y **el dato a procesar** (un mail, un comentario de un cliente, una fila de una planilla), la IA puede confundir una cosa con la otra y tomar el dato como si fuera una instrucción más.

Los **delimitadores** son marcas que separan claramente las dos partes. Los más usados son `###`, las comillas triples `"""` o etiquetas tipo `<mensaje>`. No importa cuál elijas, importa que sea consistente dentro del mismo prompt y que le digas a la IA qué hay adentro.

Nota

En una automatización esto pesa más que en un chat: el texto que entra al prompt lo escribió otra persona y vos no controlás qué dice. Delimitarlo evita que un mensaje del estilo "ignorá lo anterior y hacé otra cosa" desvíe el flujo.

El mismo caso, mal y bien

Prompt malo:

```
Clasificá este mensaje: Hola, compré un buzo la semana pasada y todavía no me llegó.
```

No tiene rol, no define las categorías posibles, no da contexto y no pide un formato. Resultado: a veces devuelve "reclamo", a veces un párrafo empático, a veces una repregunta. Para automatizar no sirve, porque el nodo siguiente no sabe qué le va a llegar.

Prompt bueno:

CÓDIGO

```
Sos un asistente de atención al cliente de una tienda de indumentaria online.

Clasificá el mensaje del cliente que está entre los delimitadores en una de estas
tres categorías: consulta_talle, reclamo_envio, otro.

Contexto: los envíos demoran 5 días hábiles. Un pedido no se considera reclamo
hasta pasados esos 5 días.

Mensaje del cliente:
###
Hola, compré un buzo la semana pasada y todavía no me llegó.
###

Devolvé únicamente la categoría, en minúscula, sin explicación ni texto adicional.
```

Este devuelve siempre `reclamo_envio`, siempre en el mismo formato, listo para que un nodo If lo use sin limpiarlo. Esa consistencia es la diferencia entre un prompt que sirve para chatear y uno que sirve para automatizar.

Dos técnicas para tener a mano

- **Zero-Shot Prompting:** es tirarse a la piletta sin saber si hay agua. Le pedimos a la IA que haga una tarea sin darle ningún ejemplo previo de cómo esperamos que la resuelva. Confiamos enteramente en su conocimiento general y en su capacidad para entender nuestra instrucción. Es lo que la mayoría de la gente hace cuando abre Gemini por primera vez. Sirve para tareas muy genéricas; el problema es que el resultado puede ser impredecible, genérico o con un formato que no nos sirve.
- **Few-Shot Prompting:** es todo lo contrario al anterior, le enseñamos con ejemplos dentro del mismo prompt. Al darle a la IA dos o tres ejemplos (esos son los famosos *shots*), ya "copia" automáticamente nuestra forma de trabajar. Entiende el tono (amable, con signos de exclamación) y la es-

estructura de la respuesta, sin necesidad de que le redactemos un párrafo gigante explicándole las reglas gramaticales que buscamos.

Repaso de conceptos del módulo

En este primer módulo sentamos las bases para comprender cómo interactuar eficazmente con la Inteligencia Artificial. Entender la diferencia entre un simple chat y el **Prompt Engineering**, así como reconocer qué es la IA generativa frente a una automatización tradicional, es el primer paso para dominar estas tecnologías. Estas tarjetas te ayudan a consolidar los pilares sobre los que vamos a construir.

01 ¿Cuál es la diferencia principal entre Inteligencia Artificial (IA) e IA generativa?

La **IA** es un campo amplio de la computación donde los sistemas realizan tareas que requieren inteligencia humana. La **IA generativa** es una rama específica capaz de crear contenido nuevo (texto, imágenes, código) a partir de instrucciones.

02 ¿Qué es realmente el Prompt Engineering y por qué va más allá de solo redactar una pregunta?

Es el proceso de diseñar, probar, refinar y optimizar las instrucciones (prompts) que se le dan a un modelo para obtener resultados más precisos, útiles y consistentes. No es solo "saber preguntar", sino un enfoque técnico e iterativo.

03 ¿Qué es un LLM y cuál es su función principal en la IA generativa?

Un LLM (*Large Language Model*) es un modelo entrenado con enormes cantidades de texto para predecir y generar lenguaje natural. Su función es procesar el contexto proporcionado para responder preguntas, resumir o crear contenido basado en patrones aprendidos.

04 Diferencias entre las técnicas de zero-shot y few-shot en el prompting.

Zero-shot es solicitar una tarea sin proporcionar ejemplos previos al modelo. **Few-shot** implica incluir uno o varios ejemplos de formato o estilo en el prompt para guiar la respuesta del modelo hacia un resultado esperado.

05 ¿Por qué no debemos confundir "automatización" con "Inteligencia Artificial"?

La automatización busca que una tarea se ejecute sola, pero no siempre requiere IA (puede ser mediante reglas fijas). La **automatización con IA** utiliza modelos para interpretar información no estructurada y tomar decisiones o generar contenido dinámico durante el proceso.

Introducción a las automatizaciones y n8n

En este segundo módulo aprendí los conceptos básicos de automatización, ROI y primeros pasos en la interfaz de n8n.

¿Qué es una automatización?

Aprendí qué son las automatizaciones y cuál es la diferencia entre una automatización lineal y una cognitiva.

Una automatización es el uso de tecnología para realizar **tareas repetitivas sin intervención humana**. Es como tener un asistente digital que sigue instrucciones precisas 24/7, moviendo datos de un lugar a otro basándose en reglas predefinidas. Las automatizaciones importan porque resuelven el dilema del crecimiento operativo. En un modelo de negocio tradicional, si los pedidos se duplican, la carga administrativa se duplica, exigiendo contratar más personal. La automatización rompe esa linealidad. Un servidor procesando 100 webhooks por minuto cuesta lo mismo y tarda el mismo tiempo que procesando 10.000. Transforma el trabajo operativo en un costo fijo, permitiendo escalar las ventas sin colapsar la logística.

A nivel corporativo y de ingeniería, la automatización no es simplemente "ahorrar tiempo". Es la **integración y orquestación de sistemas**.

¿Qué es esto de orquestación? Imaginen una orquesta sinfónica real. Tienen al violinista, al pianista y al percussionista. Cada uno es un experto en su instrumento y puede tocar de forma excelente por sí solo. Sin embargo, si cada uno toca una partitura diferente al mismo tiempo, el resultado es ruido. El **director de orquesta** es quien no toca ningún instrumento, pero da las entradas, marca los tiempos y asegura que todos trabajen juntos para crear una canción armoniosa.

En la tecnología, los "sistemas" son las diferentes aplicaciones que usa una empresa (WhatsApp, Google Sheets, Gmail, un procesador de pagos). La **orquestación** es el acto de coordinar estas herramientas independientes para que funcionen como un solo mecanismo fluido.

Automatizaciones lineales

Las automatizaciones lineales son flujos de trabajo donde cada paso se ejecuta en un orden fijo y predecible, sin que haya un modelo de IA tomando decisiones en el camino. Básicamente: "si pasa A, entonces hacé B y después C", sin ambigüedad ni interpretación.

Automatizaciones cognitivas

Las automatizaciones cognitivas son flujos de trabajo donde en uno o varios pasos se delega una tarea de interpretación, razonamiento o generación a un modelo de IA, en lugar de aplicar solo reglas fijas. La diferencia clave con las lineales es que acá el sistema no sigue únicamente instrucciones determi-

nistas, sino que en algún punto "entiende" el contenido y decide o produce algo con cierto grado de criterio.

ROI, escalabilidad y estandarización de procesos

Conocé qué es el ROI, cómo se expresa a nivel matemático y cuándo es rentable una automatización.

El **ROI (Return on Investment o Retorno de Inversión)** es una métrica financiera que evalúa la rentabilidad de una inversión en relación con su costo. En automatización, no solo se mide en dinero líquido, sino en tiempo recuperado. No mide qué tan "lindo" o complejo es el trabajo automatizado, mide **cuánta rentabilidad generó cada dólar o peso invertido en la tecnología**.

A nivel matemático, se expresa así:

$$ROI = (Beneficio\ neto / Costo\ de\ la\ inversión) \times 100$$

- **Costo de inversión:** horas de desarrollo del flujo, licencias de software, hosting, mantenimiento, etc.
- **Beneficio neto:** ahorro en horas de trabajo manual, aumento de ventas por respuestas más rápidas, reducción de costos por errores evitados (ej. evitar envíos a direcciones equivocadas).

¿**Cuándo significa que es rentable?** Es financieramente rentable cuando el ROI es un porcentaje positivo y supera el costo de oportunidad.

Nota

En automatización existe un "ROI oculto". A veces, un workflow apenas empata en términos monetarios (\$0 de ganancia directa), pero libera al dueño del negocio para que pueda dedicarse a cerrar ventas o mejorar la calidad del producto. En esos escenarios, la escalabilidad ganada hace que la inversión sea altamente rentable a largo plazo.

¿Cuándo es rentable una automatización? (Análisis de umbrales)

- **El punto de equilibrio (break-even):** es el momento exacto en el que el ahorro acumulado iguala el costo de implementación.
- **La regla del 80/20 en automatización:** el 80% de los resultados provienen del 20% de los flujos. Si automatizan algo que se hace una vez al año, el ROI siempre será negativo por el costo de mantenimiento. Automaticen procesos de **alta frecuencia y bajo juicio humano**.
- **Costo de oportunidad:** este es el argumento más fuerte. "¿Qué estaría haciendo este empleado si no estuviera copiando y pegando datos en Excel?". Si el empleado es un vendedor y pasa 2 horas copiando datos, se pierde el potencial de ventas que ese vendedor podría haber generado. El ROI no es solo lo que se ahorra, es lo que se deja de ganar por hacer tareas manuales.

¿Qué es n8n? Instancias, ejecución local y en la nube

Entendí cómo trabaja n8n, qué es una instancia y sus formas de ejecutarla, así como también en qué se relaciona JSON con n8n y cuál es su workflow.

n8n es una **herramienta** low-code y fair-code que nos **permite conectar distintas aplicaciones/servicios entre sí**, con el objetivo de automatizar tareas sin la necesidad de escribir código (aunque se puede hacer para ciertos casos específicos).

La implementación de código en una automatización es según el tipo de herramienta que estemos trabajando:

- **No code:** no se usa código en absoluto, todo es resuelto con configuraciones en interfaz visual. Si bien esto parece un beneficio, tiene una limitación, ya que dichas configuraciones se hacen en bloques prearmados. En caso de que un bloque prearmado no logre resolver lo que se requiera, estaríamos ante un problema.
- **Low code:** es un enfoque tal cual a las herramientas no-code (configuración basada en interfaz visual), con la diferencia de que en este tipo de herramientas podemos escribir código cuando sea necesario, eliminando la barrera del no-code.

Entendiendo las diferencias entre este tipo de términos, hay que asentar la idea central: cada nodo (paso de la automatización) en n8n es código ya escrito y empaquetado por los desarrolladores de n8n o su comunidad, por lo que únicamente hay que configurar parámetros mediante la interfaz gráfica. Pero nos deja esa posibilidad de implementar código en caso de ser necesario.

¿Qué es una instancia?

Es una copia en ejecución de n8n corriendo en algún lado, es decir, es el despliegue de dicha herramienta.

Formas de ejecutar una instancia:

1. **n8n Cloud:** es la versión hospedada y gestionada por el equipo propio de n8n, la cual permite ejecutar su instancia sin infraestructura a administrar. Esta manera de correr n8n es tipo SaaS (Software as a Service: modelo donde en lugar de instalar un programa, se accede a él desde un navegador).
2. **Self-hosted:** es la alternativa a n8n Cloud, ya que el encargado de la infraestructura sería uno mismo y, por lo tanto, el responsable de lo que implica. Es decir, debería tener conocimientos en redes, bases de datos, Docker, etc. Para poder hacer uso de n8n self-hosted se requiere tenerlo instalado, ya sea en Node o Docker.

¿Qué es JSON?

Cuando trabajamos con n8n, toda la información que se envía entre cada uno de los nodos e incluso todo el flujo de trabajo de n8n usa formato tipo JSON, por ende es el idioma de los datos cuando trabajamos en n8n.

JSON (JavaScript Object Notation) es un formato estándar de enviar información entre sistemas. Es como cuando enviamos un pedido, que escribimos Para: Fulano, Contenido: Pizzas. El JSON está compuesto por una etiqueta (clave) para saber qué es, y el contenido (valor).

La estructura de un JSON se escribe entre llaves y usa el formato "Llave": "Valor":

CÓDIGO

```
{  
  "nombre": "Juan",  
  "apellido": "Perez",  
  "id_cliente": 10  
}
```

En los valores podemos almacenar todo tipo de datos, ya sean strings y numbers como el ejemplo, pero también pueden ser booleanos, entre otros tipos de datos.

Importante

El medio de comunicación entre los nodos se hace mediante los paquetes JSON: eso quiere decir que cada nodo recibe y envía datos en este tipo de formato.

Workflow (el lienzo de automatización)

Un **workflow** (flujo de trabajo) es una secuencia organizada de pasos que se ejecutan automáticamente para cumplir un objetivo. En n8n, esa representación se guarda y se intercambia mediante un archivo JSON estandarizado que contiene la definición completa del flujo.

En n8n, un workflow define:

- **Cuándo comienza un proceso.**
- **Qué acciones debe realizar.**
- **Cómo se conectan esas acciones entre sí.**
- **Qué decisiones debe tomar durante la ejecución.**

En otras palabras, un workflow es el **mapa completo de una automatización**.

Importar y reutilizar templates

Comprendí qué es un template en n8n y cómo podés importarlos y reutilizarlos.

Un template es un workflow completo (con nodos, conexiones y configuración básica) que alguien creó para resolver un caso de uso específico: por ejemplo enviar un email cuando llega un nuevo lead a un formulario, sincronizar Google Sheets con Airtable, publicar automáticamente en redes sociales, etc.

n8n tiene una biblioteca pública de templates (accesible desde la propia app o desde n8n.io/workflows) donde podés buscar por categoría, integración o caso de uso.

¿Qué significa "importar" un template?

Importar un template es traer ese workflow prearmado a nuestra propia instancia de n8n, para no tener que construirlo desde cero. Generalmente podemos hacerlo de dos formas:

1. **Desde la interfaz de n8n:** en la sección de Templates dentro de la app, buscás el que te sirve y lo importás con un clic; se agrega directamente a tu lista de workflows.
2. **Manualmente con un archivo JSON:** los workflows de n8n se pueden exportar/importar como archivos `.json`. Si alguien te comparte ese archivo (o lo descargás de la web), podés importarlo desde el menú de n8n (Import from File / Import from URL).

¿Qué significa "reutilizar" templates?

Una vez importado, el template no queda fijo, podemos modificarlo libremente. Reutilizar implica:

- **Adaptar credenciales:** conectar nuestras propias cuentas (Gmail, Slack, bases de datos, etc.) en lugar de las genéricas del template.
- **Ajustar la lógica:** cambiar condiciones, campos y filtros según tu caso.
- **Guardarlo como nuestra propia plantilla interna:** si armamos un workflow que sirve para varios proyectos, podemos duplicarlo cada vez que lo necesitemos, en lugar de empezar de cero.

En resumen, los templates ahorran tiempo de armado inicial, y "reutilizarlos" es aprovechar esa base para no reinventar la lógica cada vez, solo ajustando lo que cambia entre un caso y otro.

Preentrega 1: De la idea al prompt

Diseñá (sin necesidad de construirlo funcional todavía) una automatización simple que use un paso de IA, justificando por qué vale la pena automatizarla y cómo sería el prompt que la hace funcionar.

Consigna

- 1. Elegí un proceso manual y repetitivo.** Puede ser de tu trabajo, estudio o vida diaria. Ejemplos: clasificar mails que llegan a una casilla, resumir noticias de una industria, responder preguntas frecuentes de clientes, armar un reporte semanal a partir de datos sueltos.
- 2. Justificá el ROI (2-3 líneas).** ¿Cuánto tiempo te lleva hoy hacerlo a mano? ¿Con qué frecuencia se repite? ¿Es escalable si el volumen crece? No hace falta un cálculo financiero preciso, sí una justificación razonada de por qué automatizarlo tiene sentido y por qué no.
- 3. Bocetá el flujo.** Describí (con texto o un diagrama simple, tipo lista de pasos) cómo se vería en n8n: qué dispara el flujo, qué pasos intermedios habría, dónde entra el paso de IA, y cuál es el output final. No necesita estar construido, solo pensado.
- 4. Escribí el prompt del nodo de IA.** Redactá el prompt que usarías en ese paso, aplicando lo visto sobre anatomía de un prompt efectivo: contexto claro, instrucción específica, formato de salida esperado. Un prompt vago tipo "resumime esto" no alcanza.
- 5. Reflexión corta (3-4 líneas).** ¿Qué podría salir mal? Pensá en al menos un riesgo relacionado a cómo procesa texto un LLM (alucinaciones, límites de contexto, ambigüedad) y cómo lo mitigarías en el diseño del prompt o del flujo.

Formato de entrega

Un único archivo **PDF** con las partes desarrolladas y claramente tituladas. No se evalúa prolijidad de diseño, sí coherencia entre el proceso elegido, el ROI que argumentan y el prompt que proponen. No se requiere que el flujo esté construido en n8n todavía, solo su diseño lógico y el prompt detallado.

Objetivos de aprendizaje

- Identificar procesos de negocio aptos para la automatización mediante el análisis de ROI.
- Aplicar técnicas de ingeniería de prompts para obtener resultados consistentes en una automatización.
- Evaluar riesgos críticos de los LLM como las alucinaciones para proponer medidas de mitigación en el diseño.

Criterios de aceptación

- La justificación de ROI incluye una estimación de tiempo ahorrado y frecuencia del proceso.
- El flujo descrito especifica claramente un disparador (trigger), el procesamiento de datos y la salida (output).
- El prompt incluye explícitamente rol, tarea y formato de salida deseado.
- Se identifica al menos un riesgo específico de los modelos de lenguaje y una estrategia coherente para mitigarlo.
- El entregable se presenta como un único archivo (PDF o MD) sin recurrir a formatos comprimidos.

Rúbrica de evaluación

Esta rúbrica evalúa la capacidad de identificar un proceso automatizable, justificar su valor con criterios de ROI, diseñar lógicamente un workflow en n8n, redactar un prompt útil para un nodo de IA y anticipar riesgos con estrategias de mitigación.

Criterio	Descripción	Peso
Identificación del proceso y justificación de ROI	Evalúa si elige un proceso manual realmente repetitivo y si explica con claridad por qué conviene automatizarlo, incluyendo una estimación razonable del tiempo ahorrado y su impacto en la productividad.	15%
Diseño lógico del workflow en n8n	Evalúa si describe una secuencia lógica y ordenada del flujo, incluyendo disparador, datos de entrada, procesamiento y resultado final, usando una estructura comprensible para implementar después en n8n.	40%
Ingeniería del prompt para el nodo de IA	Evalúa la calidad del prompt propuesto para n8n, considerando si incluye contexto, instrucción precisa y formato de salida esperado, con suficiente claridad para obtener respuestas útiles y consistentes.	30%
Gestión de errores y mitigación de riesgos	Evalúa si identifica un riesgo relevante del flujo de IA y propone una forma concreta de reducirlo mediante ajustes del workflow o del prompt.	10%
Estructura, claridad y cumplimiento del formato de entrega	Evalúa si el archivo final respeta las indicaciones: un único archivo PDF, con las secciones claramente tituladas, y una presentación clara que facilite la lectura y revisión.	5%

Total: 100 pts | Aprobación: 70 pts

Conceptos básicos

Aprendí qué es un nodo, cómo se conectan entre sí y por dónde viajan los datos. Además, los seis nodos que más vas a usar para transformar información y controlar el camino que recorre dentro del flujo.

Anatomía de un workflow: nodos, conexiones y flujo de datos

Conocé cómo está hecho un workflow por dentro: qué es un nodo, los tres tipos que existen según su función, cómo se lee un nodo en el canvas y qué definen las conexiones entre uno y otro.

Como vimos en el módulo anterior, un workflow es el que compone una secuencia de pasos automatizados, representado de forma visual como un diagrama de **nodos** conectados. Cada workflow resuelve una tarea: desde enviar un email cuando llega un formulario, hasta sincronizar datos entre dos sistemas.

Nodos

El nodo es la unidad mínima de trabajo dentro de un workflow. Cada nodo representa una acción, una condición o un disparador.

Tipos de nodos según su función:

- **Trigger (disparador):** inician la ejecución del workflow (webhook, evento de una app, ejecución manual). Siempre están al principio.
- **Action (acción):** ejecutan una tarea concreta (crear un registro, enviar un mensaje, etc.).
- **Transform (transformación/lógica):** procesan o modifican los datos que pasan por el workflow (Set, Function/Code, Merge, IF, Switch, etc.), sin necesariamente conectarse a un servicio externo.

Anatomía visual de un nodo:

- **Cuerpo del nodo:** ícono + nombre, identifica de qué servicio o función se trata.
- **Conectores de entrada (input):** a la izquierda, reciben datos del nodo anterior.
- **Conectores de salida (output):** a la derecha, envían datos al siguiente nodo. Algunos nodos tienen múltiples salidas, esos los vamos a ver más adelante.
- **Estado visual:** un nodo puede mostrarse como no ejecutado, ejecutado con éxito, o con error, lo cual ayuda a debuggear.

Conexiones

Las conexiones son las líneas que unen la salida de un nodo con la entrada de otro. Definen el **orden de ejecución** y el **camino que recorren los datos**.

Nodos principales en n8n

Repasá los seis nodos que más vas a usar para transformar datos y controlar el flujo —Edit Field, IF, Switch, Aggregate, Merge y Wait—: qué hace cada uno y en qué situación conviene elegirlo.

Edit Field (Set)

Qué hace: permite crear, modificar o eliminar campos dentro de cada item que pasa por el workflow. Es el nodo más usado para "dar forma" a los datos: renombrar campos, calcular valores nuevos, limpiar información innecesaria.

Cuándo usarlo:

- Cuando necesitamos dejar los datos "prolijos" antes de enviarlos a otro nodo.
- Cuando queremos armar un campo nuevo combinando otros (ej. nombre completo = nombre + apellido).
- Cuando el nodo siguiente espera una estructura de datos distinta a la que llega.

IF

Qué hace: evalúa una condición sobre cada item y lo deriva por una de dos ramas: **true** o **false**.

Cuándo usarlo:

- Cuando el workflow tiene que tomar un camino distinto según un valor (ej. ¿el monto es mayor a \$10.000?).
- Es la forma más simple de agregar lógica condicional binaria.

Cada item se evalúa individualmente: si entran 5 items, algunos pueden ir por "true" y otros por "false" en la misma ejecución.

Switch

Qué hace: es la evolución del IF cuando hay **más de dos caminos posibles**. Evalúa una condición o valor y deriva el item hacia una de varias salidas (o una salida por defecto).

Cuándo usarlo:

- Cuando hay 3 o más categorías posibles (ej. país, tipo de cliente, estado de un pedido).
- Evita anidar múltiples IF uno adentro del otro.

Aggregate

Qué hace: toma varios items separados y los **agrupa en uno solo**, generalmente combinando un campo específico en una lista, o consolidando todos los items en un array.

Cuándo usarlo:

- Cuando necesitamos mandar un resumen o lista consolidada a otro sistema (ej. un solo email con todos los pedidos del día, en vez de un email por pedido).
- Cuando el paso siguiente necesita "todos los datos juntos" y no item por item.

Merge

Qué hace: combina datos provenientes de **dos ramas distintas** del workflow en una sola salida. Es el nodo que "cierra" una bifurcación: si el workflow se separó en dos caminos, Merge es lo que los vuelve a juntar.

Cuándo usarlo:

- Cuando el workflow se dividió en ramas paralelas y necesitás reunir la información nuevamente.
- Cuando necesitás cruzar datos de dos fuentes distintas (ej. datos de un CRM + datos de una planilla).

Wait

Qué hace: pausa la ejecución del workflow durante un tiempo determinado, hasta una fecha/hora específica, o hasta que se cumpla una condición externa (como recibir un webhook).

Cuándo usarlo:

- Cuando necesitamos espaciar acciones en el tiempo (ej. enviar un recordatorio 24hs después de un evento).
- Cuando el workflow depende de una respuesta externa que puede tardar.

No "hace" nada con los datos, simplemente detiene el avance del workflow por el tiempo o condición indicada, y después continúa exactamente con los mismos datos que tenía antes de esperar.

Conceptos técnicos

En este módulo aprenderás cómo se hablan dos sistemas a través de una API, cómo se arma una petición HTTP y se lee su respuesta, y cómo se guardan las credenciales que autorizan el acceso.

¿Qué es una API? Modelo cliente-servidor y endpoints

Entendé cómo hacen dos programas para hablarse sin tocar botones: qué es una API, por qué cada nodo de n8n está usando una por dentro, y qué información viaja en cada pedido y en cada respuesta.

API significa *Application Programming Interface* (Interfaz de Programación de Aplicaciones). En criollo: es la forma en la que dos programas se hablan entre sí.

Cuando una persona usa una aplicación (por ejemplo Instagram), toca botones, ve pantallas, escribe texto. Pero cuando **un programa** necesita hablar con Instagram (por ejemplo, para publicar una foto automáticamente), no puede "tocar botones": necesita otra forma de comunicarse. Esa forma es la API.

Analogía: el mozo de un restaurante

- Vos (el cliente) no entrás a la cocina a cocinar tu propia comida.
- Le pedís al mozo lo que querés.
- El mozo lleva el pedido a la cocina, la cocina prepara la comida, y el mozo te la trae de vuelta.

La API es ese mozo: es un **intermediario** que recibe pedidos (con reglas claras de cómo pedir) y devuelve una respuesta, sin que vos necesites saber cómo funciona "la cocina" por dentro.

¿Por qué esto importa en n8n?

Cuando en n8n usamos un nodo de Gmail, Slack, Google Sheets, etc., ese nodo **por dentro está usando la API** de ese servicio. n8n simplemente le hace de interfaz visual a algo que, en el fondo, es "pedirle cosas" a la API de cada servicio.

Cuando un servicio **no tiene nodo propio** en n8n, igual se lo puede usar, mediante el nodo genérico **HTTP Request**, hablándole directamente a su API. Por eso entender qué es una API es la base para poder conectar cualquier cosa, tenga o no un nodo dedicado.

¿Qué información viaja en una API?

Cuando le "pedimos algo" a una API, generalmente le mandamos:

- **Qué queremos hacer:** leer datos, crear algo nuevo, borrar algo, etc.
- **A dónde:** una dirección específica, similar a una URL de una página web.
- **Con qué datos:** por ejemplo, si queremos crear un contacto nuevo, le mandamos nombre y email.

- **Quiénes somos:** para que el servicio sepa que tenemos permiso. Esto se llama autenticación, y lo vemos más adelante.

Y la API te responde con:

- **Si funcionó o no.**
- **Los datos pedidos,** o un mensaje de error explicando qué salió mal.

¿Qué es HTTP?

El idioma en el que se hablan las APIs: cómo se compone una petición y una respuesta, para qué sirve cada método (GET, POST, PUT, PATCH, DELETE) y qué te está diciendo el status code cuando algo falla.

HTTP (*HyperText Transfer Protocol*) es el **idioma/protocolo** que usan la mayoría de las APIs (y también los navegadores) para comunicarse por internet. Es el conjunto de reglas que define cómo se arma un pedido y cómo se arma una respuesta, para que dos sistemas distintos se entiendan sin importar en qué lenguaje de programación estén hechos.

Cuando entramos a una página web, el navegador le hace una petición HTTP al servidor de esa página, y el servidor le responde con el contenido HTTP que después se ve como una página. Con las APIs pasa exactamente lo mismo, pero en vez de devolver una página bonita, devuelven datos "crudos" (generalmente en formato JSON).

Petición (request) y respuesta (response)

Toda comunicación HTTP tiene dos partes.

Request (lo que mandamos):

- **URL:** la dirección exacta a la que le hablamos (ej. <https://api.ejemplo.com/clientes>).
- **Método:** qué acción queremos hacer.
- **Headers:** información adicional sobre el pedido (ej. en qué formato se mandan los datos, o las credenciales).
- **Body (cuerpo):** los datos que mandamos, cuando aplica (por ejemplo, al crear un cliente nuevo).

Response (lo que devuelven):

- **Status code:** un número que indica qué pasó.
- **Headers:** información adicional sobre la respuesta.
- **Body:** los datos que devuelven (generalmente en formato JSON).

Métodos HTTP más comunes

Los métodos indican **qué acción** se quiere realizar. Son la base de cómo se arma cualquier pedido a una API:

Método	Qué significa	Ejemplo de uso
GET	Pedir/leer información, sin modificar nada	Traer la lista de clientes
POST	Crear algo nuevo	Crear un cliente nuevo
PUT	Reemplazar/actualizar un recurso completo	Actualizar todos los datos de un cliente
PATCH	Actualizar parcialmente un recurso	Cambiar solo el teléfono de un cliente
DELETE	Eliminar algo	Borrar un cliente

Status codes (códigos de respuesta) más comunes

Cuando una API responde, siempre incluye un número de 3 cifras que indica el resultado:

Rango	Significado general	Ejemplos comunes
200-299	Éxito	200 OK, 201 Created
300-399	Redirección	(poco frecuente en uso diario)
400-499	Error del lado del cliente (pedimos algo mal)	400 Bad Request, 401 Unauthorized, 404 Not Found
500-599	Error del lado del servidor (falló el sistema al que le pedimos)	500 Internal Server Error

Credenciales y autenticación: API Keys y OAuth2

Cómo una API sabe quién sos y si tenés permiso: API Key, Basic Auth, Bearer Token y OAuth2. Además, cómo n8n guarda esas credenciales de forma segura y cuándo conviene usar el nodo genérico HTTP Request.

Si cualquiera pudiera pedirle datos a cualquier API sin identificarse, cualquier persona podría leer o modificar información ajena. La **autenticación** es el mecanismo que usa una API para responder la pregunta: ¿quién sos vos y tenés permiso para hacer esto?

Formas de autenticación más comunes

No todas las APIs piden lo mismo. Estas son las más habituales:

- **API Key (clave de API):** es un código único (una especie de contraseña larga) que la plataforma te da cuando creás una cuenta de desarrollador. Se manda generalmente en los headers o como parámetro de la URL. Es como una tarjeta de acceso con un código impreso: quien la tiene, entra.
- **Basic Auth:** se manda un usuario y una contraseña, codificados de una forma estándar. Es de las formas más simples y antiguas, pero menos usada hoy en servicios modernos.

- **Bearer Token:** similar a una API Key, pero generalmente es un código temporal (puede expirar) que se manda en el header Authorization. Es como una pulserita de acceso a un evento, válida por un tiempo limitado.
- **OAuth2:** es el mecanismo más robusto y el más usado por plataformas grandes (Google, Microsoft, Slack, etc.). En vez de darnos una clave fija, nos hace **iniciar sesión** con una cuenta real (ej. cuenta de Google) y nos da un permiso temporal y renovable.

¿Cómo funcionan las credenciales en n8n?

n8n tiene un sistema propio para **guardar credenciales de forma centralizada y segura**, para no tener que escribir usuarios, contraseñas o tokens directamente dentro de cada workflow.

- Se configuran una vez, en la sección de Credentials.
- Quedan encriptadas en la base de datos de n8n (no se ven en texto plano).
- Una misma credencial se puede reutilizar en varios workflows y varios nodos.
- Cada tipo de nodo (Gmail, Slack, HTTP Request, etc.) va a pedir el tipo de credencial correspondiente (API Key, OAuth2, etc.), según lo que ese servicio requiera.

¿Qué es HTTP Request y para qué sirve?

El nodo HTTP Request es un nodo genérico, no está atado a ningún servicio en particular (como sí lo está el nodo de Gmail o de Slack). Es, literalmente, la posibilidad de armar a mano una petición HTTP completa (método, URL, headers, body, autenticación) y mandarla a donde uno necesite.

¿Cuándo se usa en vez de un nodo "nativo"?

- Cuando el servicio que queremos usar **no tiene nodo propio** en n8n.
- Cuando el nodo nativo existe, pero **no cubre una funcionalidad específica** que la API sí ofrece.
- Cuando estamos probando o explorando una API por primera vez, antes de automatizar algo más grande.

Todos los nodos "de marca" (Gmail, Slack, Notion, etc.) son versiones "empaquetadas y simplificadas" de exactamente esto: peticiones HTTP armadas de antemano por los desarrolladores de n8n para que el usuario no tenga que configurarlas a mano. El nodo HTTP Request te muestra "la maquinaria" que esos otros nodos esconden.

Preentrega 2: Infraestructura de integración técnica

Construí un workflow funcional en n8n que reciba o consulte datos de una fuente externa (vía API o webhook), los procese usando al menos algunos de los nodos lógicos vistos, y entregue un resultado estructurado.

Consigna

- 1. Elegí un disparador (trigger).** Puede ser un Webhook (alguien/algo manda datos a tu workflow) o un nodo que consuma una API pública mediante una request HTTP (ejemplos: clima, noticias, tipo de cambio, un endpoint público cualquiera que no requiera credenciales complejas). No hace falta OAuth2 completo si la API elegida no lo pide: con una API Key simple alcanza para cumplir el objetivo de "autenticación".
- 2. Procesá los datos aplicando lógica de control y transformación.** Identificá qué necesita tu flujo para funcionar bien: ¿hay que filtrar datos según una condición? ¿Derivar el flujo según distintos casos? ¿Juntar información de más de una fuente? ¿Limpiar o renombrar campos antes de usarlos? ¿Esperar algo antes de continuar? Elegí los nodos que mejor resuelvan esas necesidades (de los vistos en la unidad: Wait, Merge, Aggregate, Switch, If, Edit Fields, u otros si conocés) y usalos donde realmente aporten valor al flujo, no por completar una cuota.
- 3. Mapeá la anatomía del workflow.** Antes de construir (o como documentación aparte), armá un diagrama simple o lista de pasos que muestre: trigger → nodos intermedios → output. Esto debe reflejar cómo fluyen los datos entre nodos (qué entra, qué sale de cada uno).
- 4. Entregá un output final claro.** Puede ser una respuesta HTTP (si usaste webhook), un archivo, o simplemente el JSON final visible en la última ejecución del nodo. Lo importante es que se entienda qué transformación sufrieron los datos desde que entraron hasta que salieron.
- 5. Reflexión corta (3-4 líneas).** ¿Qué pasaría si la API que consultaste cae o devuelve un error? ¿Dónde pondrías un manejo de error o un nodo If para chequear el status code? No hace falta implementarlo, solo identificar dónde iría.

Formato de entrega

Un único archivo **PDF** claro y profesional que contenga:

- Captura(s) de pantalla del workflow completo en n8n (canvas con todos los nodos conectados).
- Captura del output final (JSON o resultado visible).
- El diagrama de arquitectura de la consigna 3 y la reflexión de la consigna 5.

Objetivos de aprendizaje

- Diseñar un flujo de automatización que integre servicios externos mediante protocolos HTTP.
- Aplicar nodos de lógica y transformación para manipular estructuras de datos JSON.
- Identificar puntos críticos de falla en una integración para proponer estrategias de manejo de errores.

Criterios de aceptación

- El documento PDF incluye una captura legible del workflow completo en n8n.
- El flujo utiliza al menos un nodo de tipo HTTP Request o Webhook.
- Se evidencia el uso de al menos dos nodos de lógica/transformación (If, Merge, Edit Fields, etc.).
- El diagrama de arquitectura refleja correctamente la jerarquía de los nodos implementados.
- La reflexión identifica un punto de falla coherente con la estructura del flujo diseñado.

Rúbrica de evaluación

Esta rúbrica evalúa la capacidad de diseñar, construir y documentar un flujo técnico en n8n que integre una API, aplique lógica de control y transformación de datos, y evidencie el proceso en un documento PDF profesional.

Criterio	Descripción	Peso
Diseño del flujo e integración de la fuente de datos	Evalúa si el estudiante elige un caso de uso realista y define correctamente el recorrido del dato desde el origen hasta el destino dentro del flujo.	10%
Implementación técnica en n8n	Evalúa la construcción del workflow, incluyendo el uso correcto de un trigger, al menos dos nodos de control o transformación y el uso de expresiones para mapear datos entre nodos.	50%
Manejo de errores y reflexión técnica	Evalúa la calidad de la reflexión sobre dónde incluir un nodo If o Error Trigger para gestionar fallos de una API externa y por qué esa decisión mejora el flujo.	25%
Documentación visual y profesional del PDF	Evalúa la calidad del único PDF entregado, incluyendo diagrama de arquitectura, capturas del canvas de n8n, captura de la ejecución con JSON final y presentación general del documento.	10%
Claridad técnica y uso de transformación de datos	Evalúa si los datos están renombrados, filtrados o combinados de forma comprensible y si la salida final muestra una transformación útil y bien explicada.	5%

Total: 100 pts | Aprobación: 70 pts

Creando workflows con aplicaciones y servicios externos

En este módulo verás cómo se planifica un workflow antes de construirlo —problema de negocio, proceso, accesos, errores y mantenimiento— y cómo se arma una integración real con los servicios de Google siguiendo un caso concreto de punta a punta.

Planificar un workflow antes de construirlo: entrada, proceso y salida

La fase de diseño que te ahorra reconstruir el flujo tres veces: entender el problema de negocio, dibujar el proceso, verificar accesos y permisos, y anticipar errores, duplicados, logging y mantenimiento antes de arrastrar el primer nodo.

Antes de abrir n8n y empezar a conectar nodos, conviene pasar por una fase de diseño estratégico. Esto evita reconstruir el workflow tres veces y detectar problemas que la parte técnica no puede resolver.

Entender el problema de negocio, no solo la tarea

- ¿Qué proceso manual se está automatizando? ¿Por qué se busca automatizar ese proceso? (tiempo, errores, costo, escalabilidad).
- ¿Cuál es el resultado esperado en términos de negocio?

Diseñar el proceso a automatizar

Dibujar el flujo real, con todos sus pasos correspondientes. Eso a su vez llevaría a planificar la incorporación de herramientas/servicios externos en caso de ser necesario. Es uno de los pasos más importantes, ya que planificar previo al desarrollo permite tener un flujo más dinámico y anticipar errores o improvisaciones.

Sistemas, credenciales y permisos

- ¿Qué APIs/servicios se necesitan conectar? ¿Tienen límites de rate, cuotas, o requieren OAuth?
- Verificar que tengamos los accesos ANTES de construir, no a mitad del desarrollo.

Diseñar el manejo de errores y casos límite

- ¿Qué pasa si una API falla o responde lento?
- ¿Qué pasa con datos duplicados, vacíos o mal formateados?
- ¿Se necesitan reintentos, alertas, o un canal de revisión humana para casos ambiguos?

Pensar en idempotencia y reejecución

Si el workflow se ejecuta dos veces con el mismo dato (por un reintento o un fallo), ¿duplica registros? ¿Envía emails duplicados?

Definir logging, monitoreo y alertas

- ¿Cómo vamos a saber si el workflow falló en producción sin que alguien nos avise?
- ¿Dónde quedan los logs de ejecución? ¿Hay alertas a Slack/email si algo se rompe?

Escalabilidad y mantenimiento

- ¿Quién mantiene este workflow si nos retiramos del proyecto? ¿Está documentado?
- ¿Cómo se versionan los cambios? (n8n permite exportar JSON, útil para control de versiones en git).
- ¿El diseño modular permite reutilizar partes (sub-workflows) en otros procesos?

Conectando servicios de Google

Es muy común arrancar n8n, ver el lienzo en blanco y empezar a arrastrar nodos "a ver qué sale". Funciona para workflows chicos, pero apenas el proceso involucra datos reales, sistemas externos o algo que se va a ejecutar más de una vez, esa forma de trabajar empieza a generar problemas: workflows que hay que reconstruir desde cero, datos duplicados, procesos que fallan en silencio, o automatizaciones que nadie más que quien las armó puede mantener.

Vamos a trabajar sobre un ejemplo concreto durante todo el módulo: un workflow que completa datos faltantes en un Google Sheet consultando una fuente externa. El escenario es simple de entender pero rico en decisiones de diseño: tenemos una planilla con una columna **ID** siempre completa, y las columnas **Nombre**, **Edad** y **Cargo** vacías hasta que el workflow las completa.

CÓDIGO

```
Manual Trigger → Google Sheets (leer) → Filter → HTTP Request → Google Sheets (actualizar) → Aggregate → Gmail
```

- El **Manual Trigger** no es "el trigger más fácil", es la elección correcta para una fase de desarrollo donde no queremos que el workflow se dispare solo mientras probamos.
- El **Filter** no es un simple filtro de datos, es la solución al problema de **idempotencia**: evitar que el workflow reprocese información que ya está completa si se ejecuta más de una vez.
- El **HTTP Request** trae datos de una API que completa los IDs solicitados en el Google Sheet.
- El **Aggregate** antes del Gmail no es un capricho de orden, resuelve un problema muy concreto de cómo n8n ejecuta nodos por cada ítem, evitando que se disparen cinco correos en vez de uno.
- El **Gmail** final envía un correo donde se confirme que el proceso se completó satisfactoriamente.

Implementación del AI Agent

En este módulo comprenderás qué es un AI Agent y en qué se diferencia de una cadena LLM simple, las tres piezas que lo componen —Chat Model, Memoria y Tools— y cómo se escribe el system prompt que fija su rol, su alcance y sus límites.

¿Qué es un AI Agent? Diferencias con un nodo de IA simple

Conocé el nodo que le da criterio propio a tus flujos: qué hace un AI Agent, por qué su ciclo de razonar y actuar lo vuelve autónomo, cuándo alcanza con un Basic LLM Chain, y qué aportan sus tres sub-nodos: el Chat Model, la Memoria y las Tools.

Un AI Agent (Agente de IA) es un sistema autónomo que recibe información, toma decisiones de forma racional y actúa dentro de un entorno para cumplir un objetivo, dándonos ese lado cognitivo como habíamos tratado en módulos anteriores. En n8n, el nodo AI Agent permite conectar un modelo de lenguaje (Chat Model) y una o más herramientas (Tools), y el propio agente decide, en tiempo de ejecución, qué herramienta usar y en qué orden para resolver la tarea que se le pide.

Como bien habíamos mencionado en módulos previos, a diferencia de un flujo tradicional donde nosotros definimos paso a paso qué debe pasar, con un AI Agent es el modelo de lenguaje el que "razona" sobre la petición del usuario, decide si necesita usar una herramienta, ejecuta esa herramienta, analiza el resultado y repite el proceso hasta llegar a una respuesta final. Este ciclo de razonamiento y acción es lo que le da al agente su carácter autónomo.

¿Qué es el Basic LLM Chain (cadena LLM)?

El Basic LLM Chain es una secuencia predefinida y fija de operaciones: se arma un prompt, se envía al modelo de lenguaje y se recibe una respuesta de texto. El camino de ejecución es siempre el mismo, sin importar la entrada.

Si la tarea es simple, predecible y no necesita interactuar con el mundo exterior, usen un Basic LLM Chain. Si la tarea requiere tomar decisiones, usar herramientas externas o mantener una conversación con contexto, usen un AI Agent.

Chat Model del AI Agent

El Chat Model es el sub-nodo que se conecta a la parte inferior del nodo AI Agent y que le proporciona al agente su "cerebro": es el modelo de lenguaje (LLM) que efectivamente interpreta el mensaje del usuario, razona sobre qué hacer y redacta las respuestas. Sin un Chat Model conectado, el AI Agent no puede funcionar.

n8n no incluye un único modelo: ofrece nodos de Chat Model para conectarse a distintos proveedores, permitiendo elegir el modelo según el costo, la latencia, la privacidad de los datos o las capacidades específicas que se necesiten.

Aspectos a configurar en el Chat Model:

- **Credencial / API Key:** la clave de autenticación con el proveedor elegido (OpenAI, Anthropic, Google, etc.).
- **Modelo específico:** dentro de un mismo proveedor se puede elegir entre distintas versiones o tamaños de modelo (más rápido y económico vs. más potente).
- **Parámetros de generación:** como la temperatura (creatividad de la respuesta), el máximo de tokens de salida, entre otros.

Importante

El Chat Model resuelve el "razonamiento puntual" (entender el mensaje y decidir la próxima acción), pero por sí solo no recuerda conversaciones anteriores ni puede ejecutar acciones externas: para eso existen los sub-nodos de Memoria y de Tools.

Memoria del AI Agent

La Memoria es el sub-nodo que le permite al AI Agent recordar mensajes anteriores dentro de una misma conversación. Sin memoria, cada mensaje que llega al agente se procesa de forma aislada, como si fuera la primera vez que habla con esa persona, perdiendo todo el contexto previo.

Al conectar un nodo de memoria al AI Agent, el historial de la conversación (mensajes del usuario y respuestas del agente) se guarda y se reinyecta automáticamente en cada nueva ejecución, para que el modelo pueda entender referencias como "y el segundo que me dijiste" o "seguí con lo mismo de antes".

Puntos clave sobre la memoria:

- **Es por sesión:** la memoria conserva el contexto mientras dura una conversación (identificada normalmente por un session ID), pero no persiste entre sesiones distintas a menos que se use un almacenamiento externo.
- **Tipos de almacenamiento:** n8n ofrece memoria simple en memoria RAM del propio flujo (útil para pruebas) y también nodos de memoria respaldados por bases de datos externas para conversaciones que necesitan persistir de forma más robusta o a largo plazo.
- **Ventana de contexto:** es habitual poder limitar cuántos mensajes anteriores se conservan, para no saturar el límite de tokens del modelo ni encarecer cada llamada.

Tools del AI Agent

Las Tools (herramientas) son los sub-nodos que le dan al AI Agent la capacidad de actuar sobre su entorno: consultar información externa, ejecutar acciones o acceder a datos que el modelo de lenguaje no conoce por sí mismo. Es la diferencia principal entre "un modelo que solo conversa" y "un agente que puede resolver tareas reales".

El agente analiza la descripción de cada herramienta disponible (su nombre, para qué sirve y qué parámetros necesita) y decide, en base al mensaje del usuario y al system prompt, si conviene usarla, cuál usar y con qué datos llamarla. Esto se conoce como *tool calling*: el modelo no ejecuta la herramienta directamente, sino que indica su intención de uso y n8n se encarga de ejecutar el nodo correspondiente y devolverle el resultado al agente.

Tipos de Tools más comunes en n8n:

- **HTTP Request Tool:** permite llamar a cualquier API externa.
- **Workflow Tool:** permite que el agente ejecute otro flujo de trabajo de n8n como si fuera una herramienta, reutilizando lógica ya construida.
- **Code Tool:** permite ejecutar código JavaScript o Python personalizado para transformar datos o resolver cálculos complejos.
- **Nodos nativos de n8n como Tool:** muchas integraciones de n8n (Gmail, Google Sheets, Slack, bases de datos, etc.) se pueden usar directamente como herramientas del agente.
- **Calculator:** una herramienta simple para resolver operaciones matemáticas de forma precisa, ya que los LLM no siempre calculan bien por sí solos.

Buenas prácticas al definir Tools:

- Darle a cada herramienta un nombre y una descripción claros: es lo único que el modelo "ve" para decidir cuándo usarla.
- No conectar más herramientas de las necesarias: cuantas más tools tiene el agente, más difícil es que elija correctamente y mayor es el costo.
- Definir bien los parámetros de entrada de cada tool (tipo de dato, si son obligatorios) para que el agente los complete correctamente.
- Usar el system prompt para orientar cuándo y cómo debe usarse cada herramienta, especialmente si hay varias con funciones parecidas.

El system prompt del agente: instrucciones y límites

Las instrucciones que el agente recibe antes de escuchar al usuario: cómo definir su rol, su tono, su alcance, el formato de salida y las reglas de seguridad, y por qué el system prompt tiene prioridad sobre lo que le pidan.

Resumen del nodo AI Agent

El AI Agent es el orquestador. El Chat Model le da la capacidad de razonar. La Memoria le da contexto conversacional. Las Tools le dan la capacidad de actuar sobre el mundo exterior. La combinación de estos tres elementos es lo que transforma un simple LLM en un verdadero agente autónomo dentro de n8n. Pero ¿qué pasaría si agregamos el *system prompt*?

System Prompt

El System Prompt (o "mensaje de sistema") es el conjunto de instrucciones que se le entrega al AI Agent antes de que reciba cualquier mensaje del usuario. Funciona como la "hoja de ruta" o el reglamento interno del agente: define su rol, su personalidad, sus límites y la forma en que debe comportarse a lo largo de toda la conversación (lo que vimos en el módulo 1).

A diferencia del mensaje del usuario (que cambia en cada interacción), el system prompt se mantiene fijo durante toda la ejecución del agente y tiene mayor prioridad: el modelo intenta respetarlo por encima de lo que el usuario pida, siempre que ambas cosas entren en conflicto.

¿Qué se suele incluir en un System Prompt?

- **Rol / identidad:** quién es el agente y para qué existe (ejemplo: "Sos un asistente de soporte técnico de la empresa X").
- **Tono y estilo:** formal, cercano, técnico, con o sin emojis, en qué idioma debe responder, etc.
- **Alcance y límites:** qué temas puede tratar y cuáles debe evitar o derivar a un humano.
- **Instrucciones sobre el uso de herramientas:** en qué situaciones debe usar cada Tool disponible y en qué orden.
- **Formato de salida:** si debe responder en texto plano, en JSON, con una estructura específica, etc.
- **Reglas de seguridad:** qué no debe revelar (por ejemplo, datos sensibles, instrucciones internas) y cómo manejar pedidos fuera de política.

Ejemplo

Sos un asistente virtual de la tienda "Fulanito Shop". Respondé siempre en español y de forma amable. Tu tarea es ayudar a los clientes a consultar el estado de sus pedidos usando la herramienta "Consultar Pedido". Si no encontrás el pedido, pedile amablemente el número de orden. No inventes información sobre stock ni precios: siempre consultá la herramienta correspondiente.

En n8n, el System Prompt se configura directamente en los parámetros del nodo AI Agent (por ejemplo, en el campo "System Message"), y puede incluir expresiones dinámicas (con llaves dobles) para insertar datos que vengan de nodos anteriores del flujo, como el nombre del usuario, la fecha actual o información recuperada de una base de datos.

Preentrega 3: Planificá y construí tu primer AI Agent

Planificá un workflow bajo la lógica entrada-proceso-salida, e implementá un AI Agent o Basic LLM Chain que use un servicio de Google como parte de su proceso, con un system prompt que le dé instrucciones claras y límites definidos.

Consigna

1. Planificá antes de construir. Antes de tocar n8n, definí en un documento corto:

- **Entrada:** ¿qué dispara el workflow y qué información trae?
- **Proceso:** ¿qué tiene que decidir o hacer el agente con esa información?
- **Salida:** ¿qué resultado concreto entrega al final, y dónde queda (un mail, una fila en una hoja de cálculo, una respuesta)?

2. Justificá por qué es un AI Agent o un Basic LLM Chain. En 2-3 líneas, explicá por qué optaste por alguno de ellos: ¿el agente necesita decidir qué herramienta usar según el caso? ¿Necesita mantener contexto entre pasos? ¿Tiene que elegir entre más de una acción posible?

3. Conectá un servicio de Google al flujo. Integrá al menos un servicio de Google (Sheets, Gmail, Calendar, Drive) como parte de la entrada, el proceso o la salida del agente. Por ejemplo: el agente lee filas de una Sheet, decide qué hacer con cada una, y escribe el resultado en otra columna o manda un mail.

4. Escribí el system prompt del agente. Este es el núcleo del ejercicio. Redactá el system prompt aplicando lo visto: instrucciones claras sobre su rol, qué puede y qué no puede hacer, cómo debe comportarse ante casos ambiguos o fuera de alcance, y qué formato debe tener su respuesta o acción final.

5. Reflexión corta (3-4 líneas). ¿Qué pasaría si el agente interpreta mal una instrucción ambigua y actúa sobre un dato real (por ejemplo, en la Sheet o el mail conectado)? ¿Qué límite o chequeo agregarías al system prompt o al flujo para reducir ese riesgo?

Formato de entrega

Un único archivo **PDF**, organizado y con capturas legibles, que contenga:

- La planificación (consigna 1) y el system prompt (consigna 4).
- Captura del workflow armado en n8n con el nodo AI Agent o Basic LLM Chain y el servicio de Google conectado.
- Captura de una ejecución de prueba mostrando el resultado.
- La reflexión de la consigna 5.

Objetivos de aprendizaje

- Diseñar flujos de trabajo de automatización bajo la lógica de entrada, proceso y salida.
- Diferenciar entre implementaciones de AI Agents y Basic LLM Chains según la complejidad del caso de uso.
- Configurar integraciones funcionales entre n8n y servicios de Google Workspace.
- Redactar System Prompts efectivos que establezcan límites operativos y roles claros para un modelo de lenguaje.
- Evaluar riesgos potenciales en la ejecución de acciones automatizadas por IA.

Criterios de aceptación

- El documento PDF incluye la justificación técnica de la elección entre agente o cadena.
- El System Prompt contiene explícitamente el rol, las reglas y los límites del agente.
- La captura del workflow muestra la conexión exitosa con al menos un nodo de la suite de Google.
- Se incluye una captura de pantalla que demuestra una ejecución exitosa del flujo.
- La reflexión final identifica un riesgo real de automatización y propone una solución lógica basada en prompts o lógica de flujo.

Rúbrica de evaluación

Esta rúbrica evalúa la planificación, configuración y documentación de un AI Agent o Basic LLM Chain en n8n conectado al ecosistema de Google. Se valoran la claridad del diseño, la calidad del System Prompt, la evidencia visual del workflow, la prueba de ejecución y la reflexión sobre riesgos.

Criterio	Descripción	Peso
Diseño de la arquitectura y justificación técnica	Evalúa si el documento define con claridad la entrada, el proceso y la salida del flujo, y si justifica de forma correcta la elección entre AI Agent y Basic LLM Chain según la tarea.	30%
Calidad del System Prompt	Evalúa si el System Prompt incluye adecuadamente rol, reglas de comportamiento, límites y formato de salida, y si está redactado para guiar al agente de manera segura y útil.	30%
Evidencia visual del workflow en n8n y conexión con Google	Evalúa si la captura muestra con claridad la estructura del flujo en n8n y la conexión con al menos un servicio de Google, con nodos de IA identificables.	15%
Prueba de ejecución y evidencia del resultado	Evalúa si se muestra una ejecución real del flujo y si la captura demuestra que la IA generó un resultado visible en el nodo de salida o en el servicio de Google.	15%
Análisis de riesgos y control de seguridad	Evalúa si la respuesta explica qué mecanismo de control o instrucción adicional se añadió para evitar acciones erróneas sobre datos reales, y si la reflexión es concreta y pertinente.	10%

Total: 100 pts | Aprobación: 70 pts

Instalando IA local con Ollama

En este módulo comprenderás por qué conviene en términos de costo, privacidad e independencia, cuál es el trade-off que estás aceptando a cambio, y cómo Ollama descarga y ejecuta modelos que después n8n puede consumir.

Por qué ejecutar un modelo local: costo, privacidad e independencia

Las tres razones para bajar el modelo a tu infraestructura —dejar de pagar por token, que los datos sensibles nunca salgan de tu red y no depender de un proveedor externo— y la contrapartida honesta: en hardware de consumo los modelos son más chicos y rinden por debajo de los de punta en razonamiento complejo.

Cuando conectamos un workflow de n8n a un modelo de lenguaje, la opción por defecto suele ser una API en la nube (OpenAI, Anthropic, Google, etc.). Pero ejecutar el modelo **en nuestra propia máquina o servidor**, usando una herramienta como Ollama, es una alternativa cada vez más viable.

Las tres razones principales:

Costo

- Las APIs en la nube cobran **por token** (entrada + salida). En workflows que procesan grandes volúmenes de texto, el costo escala linealmente con el uso.
- Un modelo local no tiene costo por request: una vez descargado, las inferencias son gratuitas, el único costo es el hardware.
- Es especialmente atractivo para **prototipado y testing**: podemos iterar un workflow cientos de veces sin gastar un centavo en tokens.

Privacidad

- Con un modelo local, los datos **nunca salen de nuestra infraestructura**. Esto es crítico si el workflow procesa información sensible: historias clínicas, contratos, datos de clientes, código propietario, etc.
- Permite cumplir con regulaciones de protección de datos sin depender de los términos de servicio de un tercero.
- Es la opción natural para entornos sin conexión a internet o corporativos con políticas estrictas de manejo de información.

Independencia

- No dependemos de la disponibilidad de un proveedor externo (caídas de servicio, cambios de precio, deprecación de modelos, límites de rate).

- Podemos operar sin conexión a internet, lo cual es valioso para demos, entornos de desarrollo aislados o ubicaciones con conectividad limitada.

Importante

Contrapartida: los modelos locales que corren en hardware de consumo suelen ser más pequeños y, en tareas de razonamiento complejo, tienden a rendir por debajo de los modelos de punta en la nube (GPT-4o, Claude, Gemini). Es un trade-off entre **costo/privacidad** y **capacidad bruta del modelo**.

Ollama y descarga de modelos

Qué es Ollama y por qué no es lo mismo que un modelo: el motor que descarga, guarda y ejecuta modelos en tu computadora. Incluye qué contiene un archivo de modelo, cómo se mide su tamaño en parámetros y cuánta memoria necesita cada tramo para correr con velocidad aceptable.

Un modelo de lenguaje (como los que usan ChatGPT o Claude) es un archivo gigante con "conocimiento" entrenado. Para poder usarlo, necesitamos algo que lo cargue en la memoria de la computadora y nos deje hacerle preguntas. Eso es Ollama: un programa que **descarga, guarda y hace funcionar modelos de IA en nuestra propia computadora**, sin depender de internet ni de una empresa externa.

Es como un reproductor de música: el modelo es la canción (el archivo), y Ollama es el reproductor que la "hace sonar" y permite interactuar con ella.

¿Qué es un "modelo" en este contexto?

Es un archivo (bastante pesado, de varios GB) que contiene todo lo que la IA "aprendió" durante su entrenamiento. Hay muchos modelos distintos, hechos por distintas empresas, y cada uno tiene un **tamaño** (medido en "parámetros": cuanto más grande, más capaz pero más pesado de correr) y una **especialidad** (algunos son generalistas, otros están optimizados para código, otros para conversación, etc.).

Cómo elegir el modelo según el hardware disponible

El factor limitante de un modelo local no es solo "cuánto sabe", sino cuánta memoria (RAM/VRAM) necesita para cargarse y correr con velocidad aceptable. Como regla general, el tamaño del modelo determina el requisito de memoria:

Tamaño del modelo	RAM/VRAM necesaria	Hardware típico	Ejemplos de modelos
~1-3B parámetros	4-8 GB	Laptop sin GPU dedicada, Raspberry Pi de gama alta	Qwen2.5 1.5B/3B, Phi-3 mini, Gemma2 2B

Tamaño del modelo	RAM/VRAM necesaria	Hardware típico	Ejemplos de modelos
~7-8B parámetros	8-16 GB	Laptop con 16 GB de RAM, GPU de consumo (8 GB VRAM)	Llama 3.1/3.2 8B, Mistral 7B, Qwen2.5 7B
~13-14B parámetros	16-24 GB	Desktop con GPU media (RTX 4070/4080)	Qwen2.5 14B
~32-34B parámetros	24-40 GB	GPU de gama alta o varias GPUs	Qwen2.5 32B, Mixtral
70B+ parámetros	48 GB+ (o multi-GPU)	Servidor con GPU de datacenter (A100, H100) o varias GPUs de consumo	Llama 3.3 70B

Detalles a tener en cuenta:

1. **Ollama = motor local**, no un modelo en sí mismo.
2. **Los modelos se descargan por separado**, cada uno con sus propias características.
3. **El tamaño del modelo importa**: modelos más grandes = mejores respuestas, pero necesitan más memoria y potencia de la computadora.
4. **Ollama queda "escuchando"** en la computadora para que otras apps, como n8n, le puedan pedir respuestas.
5. **No hace falta internet** una vez que el modelo ya está descargado.

WhatsApp Business

En este módulo descubrirás qué es la WhatsApp Business Platform y qué jerarquía de cuentas de Meta hace falta para usarla, y cómo se ensambla el asistente completo —trigger, AI Agent con memoria aislada por teléfono y respuesta— con un prompt pensado para el formato móvil.

Cómo funciona la WhatsApp Business Platform (Cloud API)

Aclará de una vez las tres cosas que se confunden bajo el mismo nombre —la app personal, la app Business y la Platform— y conocé la jerarquía de cuentas de Meta que hay que atravesar para conectar un número a n8n, más los casos de uso típicos que resuelve.

Es importante distinguir tres cosas que suelen confundirse bajo el mismo nombre:

- **WhatsApp (app normal)**: la app de mensajería personal, sin API, pensada para uso individual.
- **WhatsApp Business (app)**: una versión de la app pensada para pequeños negocios, con catálogo, respuestas automáticas básicas y etiquetas. Sigue siendo una app manual, sin forma oficial de conectarla a n8n.
- **WhatsApp Business Platform (antes llamada "WhatsApp Business API")**: la solución de Meta pensada para que empresas envíen y reciban mensajes de forma **programática**, a través de una API. Esta es la que se conecta con n8n.

Cuando hablamos de "WhatsApp Business en n8n", nos referimos siempre a esta tercera opción: la API oficial de Meta.

El ecosistema detrás de la API: Meta Business

Para poder usar la Cloud API de WhatsApp Business, la cuenta no se conecta "directo", depende de una jerarquía de cuentas de Meta que conviene entender conceptualmente:

- **Meta Business Account (Business Manager)**: la cuenta que agrupa activos de una empresa (páginas de Facebook, cuentas de Instagram, cuentas de WhatsApp, etc.).
- **WhatsApp Business Account**: la cuenta específica de WhatsApp dentro del Business Manager, asociada a uno o más números de teléfono.
- **App de Meta for Developers**: es donde se generan las credenciales técnicas (tokens, IDs) que después n8n va a usar para autenticarse.
- **Número de teléfono verificado**: cada WhatsApp Business Account tiene uno o más números habilitados para enviar/recibir mensajes vía API (ese número deja de poder usarse en la app normal de WhatsApp).

Esta jerarquía es la razón por la que conectar WhatsApp a una automatización no es tan directo como, por ejemplo, conectar un webhook de un formulario: hay una capa de configuración y aprobación de por medio (Meta puede pedir verificación del negocio).

Casos de uso típicos en n8n

- **Atención al cliente automatizada:** recibir consultas, clasificarlas (con o sin IA) y responder automáticamente o derivar a un humano.
- **Notificaciones transaccionales:** confirmaciones de pedido, recordatorios de turno, avisos de envío, usando plantillas aprobadas.
- **Captura de leads:** un mensaje entrante dispara la creación de un contacto en un CRM.
- **Chatbots conversacionales:** combinando el trigger de WhatsApp con un nodo de IA (como el AI Agent visto en el módulo anterior), para sostener una conversación con memoria de contexto.
- **Encuestas y confirmaciones:** usando la operación de "esperar respuesta" para pausar el workflow hasta que el usuario conteste.

WhatsApp Business + AI Agent + Prompt Engineering

Cómo se ensambla el asistente: el AI Agent como motor de decisión, un system prompt con las reglas que exige el formato móvil, la memoria aislada por número de teléfono y la arquitectura de tres fases dentro del canvas.

El agente de IA: el motor de decisión

A diferencia de un flujo de automatización lineal tradicional, un agente de IA (implementado a través del nodo AI Agent en n8n) actúa como un orquestador dinámico que comprende el contexto.

- **Autonomía y enrutamiento:** el agente recibe el mensaje del usuario a través de WhatsApp y decide de qué acción tomar basándose en el razonamiento del modelo de lenguaje (LLM).
- **Uso de herramientas (tool calling):** si el usuario hace una pregunta general, el agente simplemente responde. Si el usuario pide consultar un pedido, revisar stock o agendar una reunión, el agente sabe que debe invocar herramientas externas (Tools conectadas a bases de datos o APIs) antes de formular su respuesta final.
- **Tolerancia a la ambigüedad:** los usuarios en WhatsApp suelen enviar mensajes cortos, con errores de tipeo o audios transcritos. El agente es capaz de inferir la intención real sin depender de palabras clave exactas.

Prompt Engineering: reglas para entornos de mensajería

El *system prompt* es el código fuente del comportamiento del asistente. Para un canal como WhatsApp, el diseño del prompt exige reglas estrictas adaptadas al formato móvil:

- **Rol y restricciones:** definir quién es el asistente, a qué negocio representa y, fundamentalmente, qué tareas tiene prohibido hacer o qué temas debe desviar.
- **Formato de salida y concisión:** las respuestas deben ser directas. Se debe instruir al LLM para generar párrafos cortos (máximo 2 o 3 líneas) y evitar bloques de texto densos que arruinan la experiencia de lectura en el teléfono.

- **Estilo visual:** forzar la utilización del formato nativo de WhatsApp, usando asteriscos para las **negritas**, guiones para crear listas legibles y emojis estratégicos para aportar calidez sin saturar el mensaje.
- **Micro-cierres (CTA):** cada interacción debe guiar al usuario. El prompt debe exigir que el agente termine sus mensajes con una pregunta clara o indicando el siguiente paso a seguir.

La memoria: persistencia y contexto (Session ID)

Para que la interacción en WhatsApp se sienta como un chat real, n8n debe inyectar el historial de la conversación en cada nueva ejecución.

Importante

El Session ID es la regla de oro para WhatsApp: para que el agente no mezcle los chats de diferentes clientes, la memoria debe estar aislada por usuario. En n8n, el identificador único (Session ID) que se mapea en el nodo de memoria debe ser siempre el número de teléfono del remitente (el valor `wa_id` o `from` que llega en el webhook de WhatsApp).

Arquitectura del flujo

La integración teórica se consolida en tres fases dentro del canvas:

- **Trigger:** nodo que escucha pasivamente los eventos entrantes de la API de WhatsApp (extrayendo el campo `text.body`).
- **Procesamiento:** el nodo AI Agent, alimentado por tres sub-nodos: el LLM (el motor lógico), la Memoria (aislada por número de teléfono) y las Tools (acciones permitidas).
- **Respuesta:** el nodo nativo de WhatsApp Business que toma la cadena de texto final generada por el agente y la dispara de vuelta al dispositivo del usuario.

Preentrega 4: Ensamblaje del asistente de WhatsApp e IA

Diseña y justificá la arquitectura de un agente conectado a WhatsApp Business, tomando una decisión informada sobre si conviene correrlo con un modelo local (Ollama) o uno en la nube, y construí una primera versión funcional del flujo.

Consigna

- 1. Definí el caso de uso del agente de WhatsApp.** Elegí un escenario concreto: atención al cliente básica, agendamiento de turnos, FAQ de un negocio, calificación de leads, etc. Describí en 2-3 líneas qué necesita resolver el agente y qué tipo de mensajes va a recibir.
- 2. Decidí: ¿modelo local o en la nube?** Con base en lo visto sobre costo, privacidad e independencia, justificá tu elección para este caso puntual. Pensalo en términos concretos: ¿el negocio maneja datos sensibles de clientes? ¿El volumen de mensajes esperado hace que el costo por request importe? ¿Necesitás depender de que un proveedor externo esté disponible? No hay una respuesta "correcta" universal, se evalúa que la decisión esté argumentada y no sea arbitraria. Si elegís modelo local, indicá qué modelo de Ollama usarías y por qué (tamaño, capacidad, para qué alcanza).
- 3. Planificá la conexión con WhatsApp Business.** Describí (con diagrama o lista de pasos) cómo entra un mensaje de WhatsApp al workflow, cómo lo procesa el agente —pasando por el nodo de AI Agent con memoria— y cómo sale la respuesta de vuelta al usuario.
- 4. Escribí el system prompt del agente.** Aplicá lo visto sobre prompt engineering: rol del agente, tono acorde al canal (WhatsApp suele ser más informal y breve que un mail), qué información tiene disponible, qué hacer si no puede resolver algo (¿deriva a un humano?), y límites claros de lo que no debe responder o prometer.
- 5. Reflexión corta (3-4 líneas).** Si mañana el volumen de mensajes se multiplica por 10, ¿tu decisión del punto 2 se sostiene? ¿Qué cambiaría en la arquitectura (local vs. nube) o en el flujo para escalar sin romper costo, privacidad o tiempos de respuesta?

Formato de entrega

Un único archivo **PDF** que contenga:

- Las consignas 1, 2 y 5 desarrolladas.
- El diagrama o lista de pasos de la consigna 3.
- El system prompt de la consigna 4.
- Capturas del prototipo de flujo armado en n8n y de su ejecución (con webhook simulando WhatsApp si no hay integración real disponible).

Objetivos de aprendizaje

- Justificar la elección entre modelos de lenguaje locales y en la nube basándose en criterios de privacidad y costo.
- Diseñar la lógica de integración entre la WhatsApp Business Platform y n8n.
- Redactar prompts de sistema efectivos y adaptados a las limitaciones y tono del canal WhatsApp.
- Planificar la escalabilidad de una solución de automatización con IA ante aumentos de tráfico.

Criterios de aceptación

- La justificación del modelo (local vs. nube) incluye al menos dos argumentos técnicos sólidos.
- El flujo de datos describe correctamente el uso de webhooks para la recepción y nodos de respuesta para el envío.
- El system prompt incluye un rol definido, restricciones de respuesta y una política de escalado a humano.
- La reflexión sobre escalabilidad aborda el impacto en los costos o en la infraestructura de hardware local.
- El documento se entrega en formato PDF y cumple con todos los puntos de la consigna.

Rúbrica de evaluación

Esta rúbrica evalúa la propuesta técnica y arquitectónica de un asistente de WhatsApp Business con IA. Se valora que el documento explique con claridad el caso de uso, justifique la infraestructura elegida, describa el flujo en n8n, redacte un system prompt adecuado para WhatsApp y analice la escalabilidad de la solución.

Criterio	Descripción	Peso
Definición del caso de uso y alcance funcional	Evalúa si el documento identifica con precisión el negocio, la función principal del bot y el tipo de mensajes que recibirá, delimitando un caso de uso realista y útil para WhatsApp.	20%
Justificación de infraestructura y elección del modelo	Evalúa si la decisión entre modelo local y nube está fundamentada con criterios de privacidad de datos, costos operativos y latencia, y si la elección es consistente con el caso de uso.	20%
Diseño del flujo en n8n y arquitectura de datos	Evalúa si el estudiante representa el recorrido del dato desde el webhook de WhatsApp hasta la respuesta final, incluyendo el AI Agent con memoria aislada por teléfono y la lógica general de integración.	25%
Ingeniería del System Prompt para WhatsApp	Evalúa la calidad del prompt del sistema, su adecuación al canal móvil, la brevedad, el tono, la orientación al objetivo y la instrucción sobre qué hacer cuando el agente no conoce una respuesta.	20%

Criterio	Descripción	Peso
Análisis de escalabilidad y preparación para crecimiento	Evalúa si el estudiante explica con realismo cómo impactaría un aumento de tráfico del 1000% en la arquitectura propuesta y qué ajustes técnicos haría para sostener la solución.	15%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 9

Proyecto Final: despliegue de agente autónomo cognitivo

Integrar todos los componentes del curso en un agente autónomo funcional desplegado.

Proyecto Final: asistente autónomo de WhatsApp desplegado

Una empresa (real, ficticia, o tu propio emprendimiento) necesita automatizar un proceso de negocio que hoy consume tiempo humano de forma repetitiva, tiene datos entrando desde más de una fuente, y podría beneficiarse de un agente de IA con criterio propio para tomar decisiones dentro de límites definidos.

Tu tarea es diseñar, justificar y construir un **sistema de automatización** que resuelva ese problema, integrando los conceptos vistos a lo largo del curso. No se trata de usar cada herramienta por separado, sino de que todas las piezas convivan en un mismo flujo con sentido de negocio.

Requisitos mínimos del sistema

El proyecto debe incluir, como mínimo, los siguientes componentes. Vos elegís cómo se combinan según el caso de uso que definas.

1. Caso de negocio con ROI justificado

- Definí el proceso manual que estás automatizando.
- Cuantificá (aunque sea de forma estimada) el tiempo/costo actual y la frecuencia del proceso.
- Justificá por qué automatizarlo escala mejor que mantenerlo manual, y en qué punto dejaría de tener sentido automatizarlo (límites del ROI).

2. Al menos dos fuentes de entrada distintas

El workflow debe recibir datos de **más de un origen**, por ejemplo alguna combinación de:

- Un webhook (formulario, WhatsApp, evento externo).
- Una API externa consultada por HTTP (con autenticación por API Key u OAuth2, según lo que la API requiera).
- Un servicio de Google (Sheets, Gmail, Calendar, Drive) como fuente de datos.

3. Lógica de control y transformación con criterio

Usá nodos **donde realmente resuelvan algo**. No se evalúa la cantidad de nodos, sino que cada uno esté ahí por una razón identificable dentro del flujo de datos (podés usar sticky notes para documentarlo).

4. Un AI Agent (no un nodo de IA simple)

El sistema debe incluir al menos un AI Agent que:

- Tenga un **system prompt** completo: rol, alcance, tono, qué puede y no puede hacer, cómo actuar ante ambigüedad o casos fuera de alcance.
- Tome una decisión real dentro del flujo (elegir una acción entre varias, clasificar, derivar, etc.), no solo generar texto.
- Esté justificado: explicá por qué este paso necesita un agente y no alcanzaba con un nodo de IA simple.

5. Decisión de modelo: local vs. nube

Elegí, para el agente principal del sistema, si conviene correr un modelo local (Ollama) o uno en la nube. Justificá con los criterios vistos: costo según volumen esperado, privacidad de los datos que maneja el proceso, e independencia de proveedores externos. Si el sistema tiene más de un paso de IA, está bien usar modelos distintos en distintos pasos, siempre que cada elección esté justificada.

6. Canal de salida orientado a un caso real

La salida del sistema debe llegar a algún lugar concreto y usable: un mensaje de WhatsApp Business, un mail, una fila actualizada en Sheets, un evento en Calendar, etc. Si usás WhatsApp Business, el system prompt del agente debe reflejar el tono y las restricciones propias de ese canal (breve, informal, con derivación a humano si corresponde).

7. Manejo de errores y casos límite

Identificá al menos dos puntos del flujo donde algo puede salir mal (una API que no responde, un dato ambiguo, el agente interpretando mal una instrucción) y mostrá cómo el diseño del workflow contempla esos casos, ya sea con nodos If que chequeen status codes, límites explícitos en el system prompt, o pasos de validación antes de ejecutar una acción sobre datos reales.

Cómo armar el repositorio de entrega

La entrega es un **repositorio** (GitHub, GitLab o similar), no un archivo suelto. Tiene que contener:

1. **El workflow exportado en .json**: en n8n, abrí tu workflow, entrá al menú de los tres puntos (arriba a la derecha) y elegí *Download*. Eso descarga el .json con toda la estructura del flujo. Subilo al repo con un nombre claro (ej. `workflow-agente-whatsapp.json`).
2. **Un README.md** con: la descripción del caso de negocio y el ROI estimado, la justificación de la elección del modelo (local vs. nube), la explicación de por qué este paso necesita un AI Agent y no alcanzaba con un nodo de IA simple, y los dos puntos del flujo donde contemplaste que algo puede fallar.
3. **El System Prompt completo** del agente, pegado en el README dentro de un bloque de código.
4. **Capturas de pantalla** del canvas del workflow y de una ejecución de prueba exitosa (podés ponerlas en una carpeta `/capturas` y referenciarlas desde el README).

Aviso

Antes de entregar, revisá que el `.json` esté subido y que no tenga credenciales dentro. n8n no exporta las credenciales en el `.json`, pero sí puede exportar valores que hayas escrito a mano en un nodo (una API Key pegada directo en un header, por ejemplo). Si ves alguna, borrala del archivo antes de subirlo.

Checklist de entrega

- El repo tiene el `.json` del workflow exportado desde n8n.
- El README explica el caso de negocio, el ROI y por qué se automatiza.
- El README justifica la elección de modelo (local vs. nube).
- El System Prompt está completo y visible en el README.
- Hay capturas del canvas y de una ejecución.
- El workflow tiene al menos dos entradas distintas, un AI Agent y un camino de manejo de errores.

Objetivos de aprendizaje

- Diseñar un flujo de automatización complejo que integre múltiples fuentes de datos y lógica condicional.
- Implementar un agente de IA autónomo con prompts de sistema optimizados para la toma de decisiones.
- Justificar la selección de modelos de lenguaje (locales vs. nube) basados en criterios de costo, privacidad y rendimiento.
- Configurar mecanismos de manejo de errores para aumentar la resiliencia de la automatización en entornos productivos.

Criterios de aceptación

- El repositorio incluye el archivo `.json` exportado del workflow funcional de n8n.
- El workflow contiene al menos dos nodos de entrada de datos distintos (ej. HTTP Request, Webhook, Google Sheets).
- El AI Agent tiene un System Prompt definido que delimita claramente sus funciones y tono.
- El README incluye una sección de justificación de ROI y una explicación de la elección del modelo (local vs. nube).
- El sistema cuenta con al menos un camino de manejo de errores o validación de datos antes de la ejecución final.

Rúbrica de evaluación

Esta rúbrica evalúa la implementación y documentación de un agente autónomo funcional en n8n para WhatsApp, integrando entradas múltiples, un AI Agent con razonamiento estructurado, un modelo de IA configurado, manejo de errores y entrega completa del workflow exportado.

Criterio	Descripción	Peso
Implementación del workflow en n8n	Valora si el repositorio contiene un workflow funcional y bien construido en n8n que integre correctamente las piezas pedidas por el ejercicio.	50%
Caso de negocio, ROI y justificación de automatización	Evalúa la calidad del análisis inicial del problema, incluyendo la descripción del proceso, el cálculo del ROI estimado y la justificación de por qué es automatizable y valioso.	10%
Configuración del AI Agent y del modelo	Evalúa si el agente de IA está configurado de manera adecuada, con un System Prompt estructurado y una elección de modelo justificada según el contexto del proyecto.	25%
Manejo de excepciones y robustez del flujo	Valora la capacidad del sistema para responder de forma controlada ante fallos, entradas inválidas o problemas con APIs y credenciales.	15%

Total: 100 pts | Aprobación: 70 pts

Glosario, análisis y cierre del curso

Un repaso transversal de los conceptos del curso: el vocabulario mínimo con el que se trabaja, tres análisis que atraviesan todo el programa, las preguntas que más aparecen y las conclusiones con las que cerrarás la cursada.

Palabras clave

- **n8n**: es una herramienta de automatización de flujos de trabajo basada en nodos que permite conectar diferentes aplicaciones y servicios sin necesidad de escribir código complejo. Funciona como un puente visual donde definís qué sucede cuando ocurre un evento específico, permitiendo automatizar tareas repetitivas.
- **API (Interfaz de Programación de Aplicaciones)**: es un conjunto de reglas que permite que dos programas o servicios diferentes se comuniquen entre sí de manera automática. Por ejemplo, cuando una aplicación móvil muestra el pronóstico del tiempo, utiliza una API para solicitar esos datos a un servidor meteorológico externo.
- **Workflow (flujo de trabajo)**: es una secuencia de pasos o tareas automatizadas que se ejecutan de manera lógica para completar un proceso. En una herramienta como n8n, un workflow comienza con un disparador y sigue una serie de nodos que procesan o transforman la información hasta alcanzar un resultado final.
- **Webhook**: es una forma en que una aplicación envía información en tiempo real a otra cada vez que ocurre un evento específico, sin que la segunda tenga que estar preguntando constantemente. Un ejemplo es cuando recibís una notificación instantánea en tu teléfono en cuanto alguien completa un formulario en tu sitio web.
- **Nodo**: es el elemento básico de construcción en los flujos de trabajo; representa una acción específica o una pieza de lógica dentro de tu automatización. Por ejemplo, puede haber un nodo dedicado exclusivamente a recibir un correo de Gmail y otro nodo destinado a enviar esos datos a una hoja de cálculo.
- **LLM (Large Language Model)**: es un tipo de inteligencia artificial entrenada con grandes cantidades de texto para comprender, generar y manipular lenguaje humano de manera coherente. Modelos como GPT de OpenAI son LLM que permiten a los agentes de IA redactar respuestas, clasificar datos o resumir documentos automáticamente.
- **Prompt Engineering**: es la técnica de diseñar y refinar las instrucciones o preguntas que le das a una IA para obtener el resultado más preciso y útil posible. Al incluir roles, contextos claros y restricciones específicas, lográs que la IA se comporte de la manera que esperás en lugar de dar respuestas genéricas.
- **Agente autónomo**: es un sistema de inteligencia artificial capaz de tomar decisiones, utilizar herramientas externas y realizar tareas de forma independiente para alcanzar un objetivo definido. A di-

ferencia de un chatbot simple, un agente puede evaluar un problema, buscar información en una base de datos y ejecutar acciones sin intervención constante del usuario.

- **JSON:** es un formato estándar y sencillo para organizar y transmitir datos de manera que tanto humanos como computadoras puedan leerlos fácilmente. Se utiliza comúnmente en las automatizaciones para estructurar la información que viaja entre diferentes servicios, pareciéndose a una lista organizada de etiquetas y valores.
- **Ollama:** es una herramienta que te permite descargar y ejecutar modelos de inteligencia artificial directamente en tu propia computadora, sin depender de servicios externos en la nube. Esto es especialmente útil para trabajar con datos sensibles que requieren privacidad total o para desarrollar aplicaciones que funcionen sin conexión a internet.

La transición de flujos lineales a agentes cognitivos

Para muchos, la automatización se resume en un simple "si pasa A, entonces hacé B". Este es el modelo lineal tradicional que hemos usado durante décadas. Sin embargo, en el mundo real, los problemas rara vez son tan ordenados. ¿Qué pasa cuando el cliente hace una pregunta ambigua? ¿O cuando el proceso requiere decidir entre tres herramientas distintas basándose en el tono de un correo? Aquí es donde el concepto de **agente cognitivo** cambia las reglas del juego.

El problema que resolvemos al pasar de flujos a agentes es la rigidez. Un flujo lineal se rompe ante lo inesperado; un agente, en cambio, utiliza un modelo de lenguaje grande (LLM) como su "cerebro" para razonar. Imaginá que tu flujo de trabajo es un tren: solo puede ir por las vías preestablecidas. Un agente es más bien un conductor de entrega con un GPS: conoce el destino, pero puede decidir qué calle tomar, esquivar el tráfico y reaccionar si una dirección está mal escrita.

Un error común es pensar que un agente es simplemente un "chatbot". No lo es. Un agente tiene **capacidad de agencia**, lo que significa que puede usar herramientas (Tools). Por ejemplo, en una empresa de e-commerce, un flujo lineal podría enviar un cupón si un usuario no compra en 24 horas. Un agente cognitivo puede leer el correo de queja del usuario, decidir si necesita un reembolso o un cupón, consultar el stock en tiempo real y redactar una respuesta personalizada, todo sin intervención humana.

Importante

La clave no es que el agente sea "inteligente" por sí solo, sino que sabe cuándo y cómo usar las herramientas que vos le das en n8n.

Entender esta distinción es vital para el programa porque n8n actúa como el sistema nervioso que conecta ese cerebro (OpenAI/Ollama) con las manos (Google Sheets, WhatsApp, APIs). Al dominar la lógica de agentes, dejás de construir "scripts" y empezás a construir "empleados digitales" capaces de manejar la incertidumbre.

El arte de la salida estructurada: por qué el JSON es el lenguaje de la libertad en IA

Cuando interactuamos con IAs como ChatGPT, solemos conformarnos con una respuesta en texto natural. Pero en el mundo de la automatización, el texto libre es un problema, no una solución. Si le pedís a una IA que extraiga los datos de una factura y te responde con un párrafo amable, tu base de datos no podrá entenderlo. Aquí es donde entra el **JSON (JavaScript Object Notation)** como el puente crítico entre la creatividad de la IA y la precisión de las máquinas.

Dominar la salida estructurada resuelve el caos de la información no estructurada. Sin JSON, tendrías que programar miles de reglas complejas para intentar "limpiar" lo que dice la IA. Con JSON, obligamos a la IA a hablar en etiquetas claras como `"precio": 150` o `"cliente": "Juan Pérez"`. Esto permite que n8n tome esos datos y los inserte instantáneamente en una celda de Google Sheets o los envíe como un mensaje de WhatsApp sin errores de formato.

- **El rol del prompt:** no basta con pedir JSON; hay que definir el esquema. Un buen prompt actúa como un molde de galletas: define la forma exacta que debe tener el dato.
- **Validación:** incluso las mejores IAs pueden alucinar. Aprender a validar que el JSON sea correcto es lo que separa a un aficionado de un experto en automatización.

Muchos estudiantes confunden el JSON con programación compleja. En realidad, es solo una forma de organizar etiquetas. Imaginá un archivador donde cada cajón tiene un nombre: eso es JSON. En este curso aprendés que no necesitás ser programador para usarlo, solo entender cómo "pedirle" a la IA que llene los cajones correctos. Al final del programa vas a ver que el JSON es lo que permite que tu asistente de WhatsApp no solo hable, sino que realmente "ejecute" acciones precisas en tus otros sistemas.

IA local vs. cloud: el equilibrio entre privacidad, costo y potencia con Ollama

En la fiebre de la IA, la mayoría corre hacia soluciones en la nube como OpenAI. Sin embargo, para un profesional o una empresa, esto plantea tres preguntas críticas: ¿cuánto va a costar esto a escala? ¿Qué pasa con la privacidad de mis datos sensibles? ¿Y si no tengo internet? La respuesta a este dilema es la **IA local mediante Ollama**.

Implementar modelos locales como Llama 3 o Mistral dentro de tu propia infraestructura (usando Docker y Ollama) resuelve el problema de la dependencia externa. Imaginá que estás procesando datos médicos o financieros de clientes. Enviar eso a la nube podría violar normativas de privacidad. Al usar Ollama integrado con n8n, el dato nunca sale de tu computadora o servidor. Es como tener una caja fuerte inteligente en casa en lugar de alquilar un espacio en un banco público.

Comparativa rápida: cloud vs. local

- **Cloud (OpenAI/Claude):** más inteligente, requiere suscripción o pago por uso, mayor latencia, menor privacidad.
- **Local (Ollama):** costo cero por ejecución, privacidad total, funciona offline, requiere hardware (GPU/RAM) decente.

Un error frecuente es pensar que la IA local es "inferior". Si bien los modelos gigantes en la nube son más potentes para razonamientos abstractos, un modelo local bien configurado es excepcional para tareas específicas de automatización, como clasificar correos o resumir documentos. En este programa aprendés a no depender de una sola opción: vas a saber cuándo usar la potencia de la nube para el cerebro principal de tu agente y cuándo usar Ollama para tareas repetitivas y privadas, optimizando tus costos y protegiendo tu activo más valioso, la información.

Preguntas frecuentes

¿Necesito aprender programación avanzada para tener éxito en este curso?

Muchos estudiantes temen que, al automatizar procesos, deban convertirse en ingenieros de software. Sin embargo, el objetivo de n8n es la automatización "low-code". Lo que realmente necesitás es desarrollar un pensamiento lógico: aprender a descomponer un problema complejo en pasos pequeños y secuenciales. En lugar de escribir código desde cero, vas a aprender a gestionar estructuras de datos como JSON y a entender cómo se comunican las aplicaciones mediante APIs. Es como construir con piezas de Lego: no necesitás fabricar las piezas, solo saber cómo encajarlas para crear una estructura funcional.

Nota

El pensamiento lógico y la atención al detalle te llevarán mucho más lejos que memorizar sintaxis de programación.

¿Cuál es la diferencia real entre un prompt simple y un agente de IA?

La confusión común es pensar que cualquier interacción con un modelo de lenguaje es un agente. Un prompt simple es una instrucción directa: hacés una pregunta, la IA responde y ahí termina el proceso. Un agente de IA, en cambio, tiene "capacidad de agencia". Esto significa que puede usar herramientas (tools) para investigar, consultar una base de datos en Google Sheets o realizar cálculos antes de darte una respuesta. Pensá en el prompt simple como un asistente que solo conoce lo que leyó ayer, mientras que el agente es un colaborador que puede ir a la biblioteca, buscar el libro específico y redactar un informe basado en hallazgos reales.

¿Por qué debería usar un modelo local con Ollama si ya tengo acceso a OpenAI?

La elección entre modelos en la nube y modelos locales suele basarse en tres pilares: privacidad, costos y latencia. Muchos estudiantes asumen que la nube siempre es superior, pero cuando trabajás con

datos sensibles de clientes o empresas, enviar esa información a servidores externos puede violar políticas de privacidad. Ollama te permite ejecutar modelos potentes localmente, lo que garantiza que tus datos nunca salgan de tu entorno controlado. Además, te ahorra el costo por cada token procesado.

Nota

Utilizá modelos locales para tareas de procesamiento de datos internos o de alta privacidad, y reservá las APIs potentes de la nube para tareas que requieran razonamiento creativo de alto nivel.

¿Cómo puedo practicar sin romper nada ni gastar dinero en servicios caros?

La mejor estrategia es construir un "entorno de pruebas" o sandbox. No intentes automatizar el flujo crítico de tu negocio el primer día. Comenzá con flujos simples: por ejemplo, una automatización que tome un mensaje de un formulario, lo procese con una IA para resumirlo y lo guarde en una hoja de cálculo personal. Además, herramientas como n8n permiten trabajar localmente con Docker, lo que significa que podés experimentar infinitamente sin costos de infraestructura. La práctica constante mediante la resolución de problemas cotidianos —como organizar tus correos o clasificar tus archivos— es la forma más rápida de ganar confianza técnica sin riesgos reales.

¿Es obligatorio el uso de WhatsApp Business API para integrar IA en WhatsApp?

Muchos entusiastas intentan usar métodos no oficiales (como bibliotecas de terceros que "hackean" la web de WhatsApp), pero estos suelen ser inestables y corren el riesgo de bloquear tu número. En este curso nos enfocamos exclusivamente en la Meta Cloud API. ¿Por qué? Porque es el camino profesional y escalable. Aunque requiere una configuración inicial un poco más técnica, te garantiza que tu asistente no dejará de funcionar de la noche a la mañana por una actualización de Meta. Aprender a manejar la documentación oficial de Meta es una habilidad de alto valor que te diferenciará de quienes usan soluciones "caseras" y poco confiables.

¿Qué hago si mi flujo falla y no entiendo por qué?

El debugging es, en realidad, el 70% del trabajo de un experto en automatización. Los estudiantes suelen frustrarse cuando un nodo da error, pero la clave es aprender a leer la consola y los datos de entrada/salida de cada nodo. n8n tiene un inspector de datos excelente. Si un flujo falla, no lo mires como un problema global, sino como una cadena de información rota: revisá en qué punto específico los datos dejaron de tener el formato esperado (usualmente un error de mapeo JSON).

Nota

Utilizá siempre nodos de "Error Trigger" o notificaciones de fallo para que el sistema te avise proactivamente cuando algo sale mal, en lugar de enterarte cuando el cliente te lo reclame.

Conclusiones

De la automatización lineal a la orquestación cognitiva

Completaste la transición desde la simple ejecución de tareas preprogramadas hacia la orquestación de sistemas inteligentes. Mientras que las automatizaciones tradicionales (n8n, Google Sheets, APIs) actúan como el sistema nervioso, la integración de LLM y agentes de IA funciona como el cerebro capaz de tomar decisiones no deterministas.

Importante

El valor profesional reside ahora en saber cuándo aplicar lógica rígida (si ocurre X, hacé Y) frente a lógica probabilística (analizá X y decidí la mejor acción). Esta distinción te permite construir flujos que no solo operan, sino que razonan.

Ahora tenés la capacidad de transformar procesos manuales tediosos en ecosistemas autónomos que gestionan excepciones y aprenden del contexto, una competencia de alto impacto que redefine el rol del profesional moderno en la era de la IA.

Arquitectura de sistemas resilientes y escalables

A lo largo del programa aprendiste que una automatización exitosa no es solo la que funciona bajo condiciones ideales, sino la que sabe fallar y recuperarse. Al combinar el uso de sub-workflows, reinicios automáticos, gestión de estados y logs de depuración, adoptaste una mentalidad de ingeniería de sistemas.

- **Resiliencia:** la implementación de "fail safe" asegura la continuidad operativa incluso ante caídas de servicios externos.
- **Modularidad:** el uso de sub-workflows evita que tus automatizaciones se conviertan en "espagueti", facilitando el mantenimiento a largo plazo.

Este enfoque te permite escalar soluciones desde prototipos simples hasta sistemas robustos que soportan las demandas de un entorno empresarial profesional.

La soberanía tecnológica: integrando IA privada y pública

Dominar la integración entre servicios en la nube (OpenAI, Meta Cloud API) y modelos locales (Ollama) te otorga una ventaja estratégica única: la capacidad de elegir la herramienta adecuada según los requisitos de privacidad, costos y latencia.

Nota

No todo necesita estar en la nube: aprender a desplegar IA localmente te permite procesar datos sensibles de clientes cumpliendo normativas estrictas mientras mantenés el control total sobre tu infraestructura.

Ahora no sos solo un usuario de herramientas externas, sino un arquitecto capaz de diseñar soluciones híbridas que equilibran la potencia de los grandes modelos con la soberanía de los datos propios, un activo crítico en cualquier sector que maneje información confidencial.

El futuro del profesional autónomo: tu nuevo stack

Al integrar WhatsApp Business, APIs REST y agentes cognitivos, adquiriste un superpoder: la capacidad de cerrar el ciclo entre la intención y la ejecución. Antes necesitabas desarrolladores para crear integraciones; hoy podés desplegar un asistente completo que atiende consultas, procesa datos en hojas de cálculo y actúa según las necesidades del negocio en tiempo real. Tu transformación profesional consiste en haber pasado de ser un consumidor de software a un creador de soluciones. La automatización ya no es un proyecto lateral, es el lenguaje con el que vas a optimizar cualquier rol profesional en el futuro, permitiéndote delegar la ejecución repetitiva a tus "empleados digitales" y enfocarte en el diseño estratégico y la innovación.